

Knowledge graph usage metadata: Insights from SPARQL log analysis

Maryam Mohammadi ^{a,b,*}, Chang sun ^{a,b}, Remzi Celebi ^{a,b}, Christopher Brewster ^{a,b,c} and Michel Dumontier ^{a,b}

^a *Institute of Data Science, Faculty of Science and Engineering, Maastricht University, The Netherlands*

^b *Department of Advanced Computing Sciences, Faculty of Science and Engineering, Maastricht University, The Netherlands*

^c *Data Science Group, TNO, Soesterberg, The Netherlands*

Abstract. Knowledge Graphs (KGs) are key technologies that enable enhanced understanding, knowledge representation, reasoning, and interpretation of complex data. The use of RDF KGs relies heavily on SPARQL queries for knowledge retrieval and manipulation. Analyzing SPARQL query logs can provide valuable insights into KG usage, revealing patterns in user behavior and interactions with the data. Prior studies have analyzed these logs in terms of their syntax and structure, but little is known about what parts of a KG are queried by users. This paper introduces content-based methods for analyzing KG usage, specifically examining the extent to which SPARQL queries cover the KG schema over defined time periods. We examine organic and robotic queries of Wikidata and Bio2RDF. Raw schema coverage is higher for robotic queries (60–97%) than organic ones (typically 13–21%), but because the two query types differ greatly in volume, we control for sampling effort using individual-based rarefaction. ~~This reveals a KG-dependent picture: on the compact, specialized~~ The outcome differs between the two: on Bio2RDF the broader robotic coverage is a genuine behavioral difference persists at equal effort (robotic exercises $\sim 1.8\times$ more of the schema at equal effort), whereas on ~~the very large~~ Wikidata the gap is largely ~~a sampling-size artifact attributable to sampling size~~. Crucially, coverage is not directly comparable across the two: Bio2RDF has an OWL-style schema (classes with instances), whereas Wikidata is item-based, and we find that a large share of the items counted as queried “classes” — about 30% of those in human queries — are in fact referenced as property values, not as classes. Comparing query demand against KG supply exposes a structural inversion: the most heavily instantiated Wikidata classes (e.g. geographic features) are rarely queried, while demand concentrates on a small, intuitive core. A third KG, DBpedia, ~~confirms~~ indicates that this value-position effect ~~tracks~~ is associated with the modeling style (item-based vs. OWL) rather than with KG size. Both datasets exhibit a sharp long-tailed decline in usage frequency: usage is highly concentrated (Gini ≥ 0.90 everywhere, with a fraction of a percent to a few percent of used elements accounting for 80% of all references), so a high coverage figure reflects breadth of reach rather than even demand. We use statistical assessments —, validated across seven equal-length intervals —, to track shifts in schema-element usage across KG versions, query types, and log intervals. Finally, we show the usage metadata has concrete predictive value: a type-suggestion ranking learned from one year of logs captures most of the next year’s query demand and substantially outperforms a ranking by KG content. Our work sheds light on what is and is not queried in a KG, surfaces the mismatch between KG content and query demand, and provides guidance for documentation, schema design, and the interpretation of usage metrics on item-based KGs.

Keywords: Usage Metadata, RDF Knowledge Graph, SPARQL Query Logs

*Corresponding author. E-mail: m.mohammadi@maastrichtuniversity.nl.

1. Introduction

Knowledge Graphs (KGs) are widely used as they are powerful representations of complex data. By organizing data into graph structures and ontologies, KGs facilitate semantic data integration, advanced search capabilities, and building neuro-symbolic models [1, 22]. The Resource Description Framework (RDF) is a common model to implement a semantic KG. RDF KGs represent data as triples (subject-predicate-object), which allows for flexible and machine-readable descriptions of entities and their relationships. This standardized format is widely used for Linked Data on the Web [2]. SPARQL endpoints enable programmatic interaction with RDF KGs using the SPARQL language, and these interactions can be stored in a web server. Analyzing the SPARQL query logs offers insight into what users are interested in and the manner in which they access desired information [5], revealing patterns in computational demands, reliability, performance, user behavior, and data modeling decisions [3, 6, 13, 16, 17, 19, 23, 25, 26]. These insights constitute usage metadata about the KG, complementing conventional descriptions of its structure and content with evidence of how it is accessed in practice.

Throughout this paper we use three terms consistently. A *type* (or class) is a resource used to classify instances, i.e., the object of a typing predicate such as `rdf:type/wdt:P31` or a subclass predicate such as `rdfs:subClassOf/wdt:P279`. A *predicate* is the property in a triple (s, p, o) . We refer to the set of distinct types and predicates of a KG as its *schema elements*. An *individual* is an instance, i.e., a non-schema resource (for example, a specific drug or person). ~~We refer to the set of distinct types and predicates of a KG as its *schema elements*.~~ Finally, we study *syntactic-explicit* usage: a schema element is counted as used when its IRI is explicitly written in a query. We use this term in deliberate contrast to a *semantic* (effective) notion of usage ~~that, which~~ would additionally credit the elements a query reaches only at evaluation time—, through inference or property-path traversal—, without naming them. We adopt *syntactic-explicit* usage throughout and argue (Sections 3.4 and 5) that it is both the appropriate primitive for our motivating applications (documentation, interface design, autocomplete, deprecation) and the ~~only well-behaved one~~ better behaved of the two on an item-based KG; characterising semantic usage is future work.

Previous studies on SPARQL query logs have primarily focused on the syntax and structure of these queries. These studies have analyzed keyword usage (e.g., count of SELECT, ASK, UNION, OPTIONAL) and query shapes (e.g., chain, tree, cycle), and their relation to evaluation complexity [8, 10, 27, 29]. More recent work has begun to examine *which* schema elements are queried: Bonifati et al. [7] analyze predicate usage in Wikidata query logs (including an interactive property sunburst), and Hammerer and Martens [12] catalog the Wikidata IRIs used for transitive (property-path) navigation. Asprino et al. [3] employed a content-centric approach to summarize query logs and identified query templates that qualitatively characterize user queries, revealing patterns and contexts in which different types, relations, and instances are used. What is still missing, and what we provide, is a *quantification* of how much of a KG's schema is covered by queries, of the usage frequency of each schema element, and of how this coverage shifts over time and between organic and robotic query types across two contrasting KGs. This study explores ways to better understand the usage of KGs through the analysis of SPARQL query logs. We present novel methods to quantify schema (types and ~~relations~~ predicates) usage, as well as to measure the frequency and the extent of that usage. Our research seeks to answer two main questions: (1) How can we quantify the usage of a KG? and (2) Do frequency distribution and ranking analyses of used KG elements reveal usage patterns over different log intervals or across organic (human-driven) and robotic (bot-driven) query types?

Previous work [20] reported that most of the schema (types and ~~relations~~ predicates) of the Bio2RDF KG was referred to within the SPARQL query logs over 30 months of queries. However, this work provided only a superficial view of user interaction with the KG, lacking deeper insights into the extent to which each schema element is accessed by users as well as differentiating between human and robotic interactions.

In this study, we extend prior work [20] to scrutinize the extent to which each schema element is included in user SPARQL queries while differentiating between organic and robotic queries from two KGs: Bio2RDF and Wikidata. ~~The output of this analysis can be termed "usage metadata". The method produces novel~~ method produces usage metadata that offers practical insight into the extent to which KG is used by SPARQL users. Such information may be useful in understanding what users are querying in the KG and for targeted optimizations toward enhanced user engagement and operational efficiency. An innovative aspect of our approach is that it focuses on the triple

patterns in SPARQL queries so as to focus on the types, relations, and individuals referred to in the queries. The main contributions of this paper, relative to preliminary work [20], are as follows:

- **A robust pipeline-corrected procedure for recovering schema elements from raw query logs.** Raw logged-query strings are frequently unparseable (missing prefix declarations, URL-encoding, appended HTTP parameters). We describe a preprocessing pipeline (prefix injection, URL-decoding, parameter removal, placeholder substitution, and variable standardization) that makes the logged queries parseable and aligns the schema extracted from the query side with the schema extracted from the KG side over a consistent reference set of subgraphs. Our contribution here is a correction to how appended HTTP query-string parameters are removed: rather than matching a fixed list of known parameter keywords, we strip any terminal run of `&key=value` segments, anchored to the end of the query string so that IRIs containing `&` mid-query are unaffected. We also perform this normalization before deduplication, so that queries differing only in incidental HTTP artifacts collapse to one unique query (Sections 3.3–3.4).
- **Identification and analysis of usage patterns in query logs.** We compute schema-element frequency distributions and conduct statistical assessments (Shapiro–Wilk, Wilcoxon signed-rank, Spearman’s rank correlation) to evaluate the consistency of, and changes in, schema-element usage in both Wikidata and Bio2RDF across KG versions, query types, and log intervals. We further quantify-introduce measures of how concentrated usage is (Gini, evenness, and Pareto statistics; Section 4.9.2), give evidence (with stated caveats) that newly added schema elements are rarely queried soon after introduction, and show that rare classes are reached through less-popular a procedure for relating schema-element introduction to subsequent query adoption, and a predicate co-occurrence grouping that characterises the kinds of queries through which classes are reached (Section 4.9.7).
- **A size-controlled comparison of organic vs. robotic coverage at equal sampling effort.** Because logs of different query types differ greatly in size a larger log mechanically has more opportunities to name a schema element, raw coverage differences can be sampling-size-between logs of different sizes can be sampling artifacts. We add an individual-based rarefaction analysis (and Chao1 richness estimation) that compares schema coverage at equal sampling effort, disentangling genuine behavioral differences from differences in query volume sampling effort, where effort is the number of observed schema-element occurrences rather than the number of queries. This tests whether a coverage difference persists once the number of observed occurrences is held equal, and so separates the part of the difference attributable to sample size from the part that is not.
- **What “schema coverage” means for an item-based KG, and a content–demand mismatch analysis.** Our coverage metric was first designed for Bio2RDF, whose OWL-style schema has classes with instances. Extending the study to Wikidata exposed that this assumption does not transfer: Wikidata is item-based, and we show that a large fraction of the items counted as queried “classes” are actually referenced as property values, not as classes. We separate class-position from value-position usage, and we compare query demand against KG supply (instances per class), revealing a structural inversion between what Wikidata contains and what users query. A third KG, DBpedia, confirms-indicates that this value-position inflation tracks-is associated with the schema model (item-based vs. OWL-style) rather than with KG size, while the content–demand inversion generalizes-across-is present under both.
- **A forecasting demonstration of downstream utility.** We show that the usage metadata has concrete predictive value: a type-suggestion ranking learned from 2017 logs captures most of the 2018 query demand, and substantially outperforms a ranking by KG content (instances per class), which wastes its top suggestions on bulk-imported but unqueried classes (Section 4.10.1).
- **Demonstrated applicability to additional logs.** We apply the methods to new intervals of Bio2RDF and Wikidata logs across two KGs of very different scale and style.

The organic/robotic distinction itself is not our contribution: it was introduced for Wikidata by Malyshev et al. and Bielefeldt et al. [5, 18], and the Wikidata logs we use are already partitioned accordingly. Our only addition on this point is to apply the same user-agent classification algorithm of [5] to the Bio2RDF logs—the 2019 logs and, from the raw server log, the 2013-era traffic (Section 4.7.1)—so that all analyses can be reported separately for organic and robotic queries on both KGs. The usage metadata from the proposed methods includes multiple components:

a percentage value quantifying KG's overall usage in terms of its schema elements, the usage frequency of each element, and how usage changes over different log intervals and across two query types (organic and robotic).

The remainder of the paper is organized as follows: Section 2 reviews state-of-the-art methods of SPARQL query log analysis. Section 3 describes the proposed method. Section 4 presents the results, Section 5 offers a discussion of the findings and potential future directions. Finally, Section 6 concludes with a summary.

2. Related Work

Query logs capture user interactions with a system, and their analysis may provide insight into both what users are interested in and how they approach obtaining those results. Prior work has focused on SPARQL log dataset creation, distinguishing between human and robotic queries, and query log summarization.

Query Log Datasets. Studies [27, 29] introduce the Linked SPARQL Queries Dataset (LSQ), a public, openly accessible dataset of SPARQL queries extracted from KG endpoints. LSQ facilitates reuse by addressing the issue of legal agreements that limit access to raw query logs. It standardizes the format of logs, making them accessible for analysis. In this study, we use the LSQ SPARQL endpoint to access the Bio2RDF and Wikidata logs.

Organic vs. Robotic Queries. Bielefeldt et al. [5, 18] introduced the concept of organic and robotic SPARQL queries. They proposed a method to partition the Wikidata query logs they studied (on the order of hundreds of millions of queries; the live Wikidata Query Service has since logged far more) into organic and robotic queries. Robotic queries are generated by software tools or bots, while organic queries are generated by humans using browsers. Organic queries reflect end-user behavior, robotic queries may not directly capture user needs but often address backend requirements (e.g., serving internal or analytical purposes) or support application development. By distinguishing these types, the authors identified patterns in human usage and noted that robotic traffic of Wikidata displayed high volatility and unpredictability. We classify Bio2RDF queries into organic and robotic using the method in [18] and analyze organic and robotic queries of Wikidata and Bio2RDF.

Content-Centric Analysis. Previous work by Asprino et al. [3] introduced a content-centric method to analyze LSQ logs. Their method, termed query log summarization, categorizes SPARQL queries into common templates based on their content. By examining the ten most executed templates for each log, they identified common usage patterns and the prevalence of queries originating from a single code source. They also explored template relationships, uncovering evidence that related templates often participate in common processes and are frequently executed by similar sets of hosts. Additionally, their study highlighted relationships between different queries across various logs, indicating a systematic, automated approach to data querying. This work provides qualitative insight into KG usage but does not quantify the usage of the KG content.

Closest to our content focus, Bonifati et al. [7] quantify predicate usage in Wikidata query logs and Hammerer and Martens [12] enumerate the IRIs used in property-path (transitive) navigation. Our work complements these by quantifying coverage over the *full* schema (both types and predicates), reporting it as a single coverage percentage per (log, KG-version) pair, and tracking how it shifts over time and between organic and robotic queries. Although existing studies are valuable for analyzing KG usage, they do not provide a quantitative measure of the extent to which the KG schema as a whole is queried by users. Identifying the usage patterns may help optimize the KG development by guiding efforts, such as maintenance, toward the most frequently used or queried elements. This improves user experience by ensuring high-demand data is easily accessible and reliable, and reduces costs by allocating resources more efficiently and potentially deprecating underused elements. This paper presents an approach for quantifying SPARQL schema coverage and analyzing KG content usage.

3. Method

The workflow comprises six main steps as illustrated in Figure 1, beginning with retrieving SPARQL logs and downloading KG versions that approximately align with query log periods. The second step involves extracting schema elements from the KG. Next, we clean the SPARQL logs (step 3) in preparation for extracting the schema elements used within these logs (step 4). The fifth step involves calculating the SPARQL schema coverage, to assess

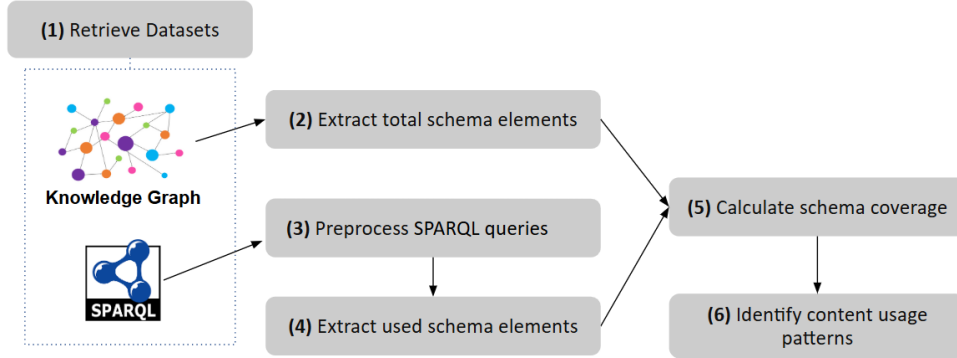


Fig. 1. Workflow of the proposed method

how well the queries in the logs represent the KG’s schema. Finally, we perform various analyses to discover usage patterns. Details of each step are described in the following subsections.

3.1. Datasets

RDF KGs. Given that our analysis spans a period of Wikidata SPARQL query logs from 2017 to 2018, we downloaded a version of Wikidata from 2017 and 2018, as these are the closest versions available to the log period. This ensures maximum alignment of schema and query logs. We use both versions to investigate how the SPARQL schema coverage changes with the evolution of the Wikidata KG. We downloaded two versions of Wikidata from the Internet Archive¹, and hosted them on Blazegraph on a local server.

For Bio2RDF KG [4], the analysis covers two periods of SPARQL query logs: one beginning in 2013 and the other in 2019. However, due to the unavailability of datasets from these years, we instead use the current 2024 public endpoint². Because a live endpoint drifts, the exact schema we extracted from it is not reproducible from the endpoint alone; we therefore *freeze* the extracted schema (types and predicates) as a released artifact, and it  not the live endpoint  is the reproducible reference for all Bio2RDF coverage figures.

SPARQL Query Logs. The query logs are primarily retrieved from the Linked SPARQL Queries Dataset (LSQ) 2.0 endpoint³ using the query illustrated in Listing 1. This query extracts the text and execution time of queries from 24 datasets in LSQ 2.0 [29], including 23 Bio2RDF datasets and one Wikidata query dataset. Furthermore, we downloaded Bio2RDF logs from Dumontier’s lab repository⁴ and Wikidata queries (all queries and organic queries for interval 1 and interval 7) from the International Center for Computational Logic⁵.

Among the many KGs available through LSQ, we selected Bio2RDF for two reasons. First, it is a domain-specific (life-sciences) KG with a compact, OWL-style schema, which contrasts sharply with the broad, general-purpose, and very large schema of Wikidata; comparing the two lets us study usage across two extremes of schema scale and design. Second, large Bio2RDF logs are available to us, including logs that retain user-agent information and therefore permit the organic/robotic split. The number of queries per dataset is substantial (ranging from ~0.2 million to ~81 million; full counts are reported in Tables 1 and 3); ~~and we relate coverage to this sampling effort explicitly in Section 4.~~

The interval boundaries are not chosen by us; they are inherited from the upstream releases. For Wikidata, intervals 1 and 7 are the two intervals of the ICCL/Dresden release that provide the organic/robotic partition together with sufficient volume, and they bracket an approximately one-year span (2017 vs. 2018) that aligns with the 2017

¹Wikidata version 2017: <https://archive.org/download/wikibase-wikidatawiki-20170821> and Wikidata version 2018: <https://archive.org/download/wikibase-wikidatawiki-20180205>

²<https://Bio2RDF.org/sparql/>

³<https://lsq.data.dice-research.org/sparql>

⁴<https://download.dumontierlab.com/Bio2RDF/logs/>

⁵https://iccl.inf.tu-dresden.de/web/Wikidata_SPARQL_Logs/en

and 2018 KG dumps we host. For Bio2RDF, the 2013 and 2019 periods correspond to the two log dumps available (LSQ and the Dumontier-lab repository, respectively). Because these periods differ in length, all frequency analyses use *time-normalized* counts (Section 3.6.1), where one month is defined as a fixed 30-day period and a log's duration in months is computed as (end date – start date)/30 days.

Listing 1: The templated SPARQL query against the LSQ SPARQL endpoint to retrieve SPARQL queries for selected datasets.

```

1 PREFIX lsqv: <http://lsq.aksw.org/vocab#>
2 PREFIX prov: <http://www.w3.org/ns/prov#>
3
4 SELECT DISTINCT ?text ?timeStamp
5
6 FROM <http://lsq.aksw.org/{dataset_name}>
7
8 WHERE {
9   ?query    lsqv:text           ?text .
10  ?query    lsqv:hasRemoteExec ?re .
11  ?re       prov:atTime         ?timeStamp .
12 }
```

We implemented the method in [5, 18] to classify queries as organic or robotic based on the agent who executed the query. The Bio2RDF log2019 dataset includes user agent information, enabling us to generate the Bio2RDF organic and robotic log2019 subsets. The criteria used for classification are as follows:

Organic: If the user agent string contains indicators commonly associated with web browsers, such as "Mozilla," "Chrome," "Safari," "Firefox," "Edge," or "Opera," it was classified as organic. These are typically representative of human users.

Robotic: Entries whose user agent lacks browser indicators were classified as robotic. In practice these agents carry terms such as "Apache," "HttpClient," "Crawler," "sparqlwrapper," "python," or "Java," which are linked to automated scripts or bots; however, the operational rule is the complement of the organic test (any non-browser, non-empty agent), so that agents not matching a fixed keyword list are still treated as robotic rather than discarded.

No agent: A small number of entries carry no user-agent string and cannot be assigned to either class; they are excluded from the organic/robotic split.

3.2. Extracting Total Schema Elements

In this study, the *schema* of a KG is the set of its distinct *types* (classes) and *predicates*. A *schema element* is one such type or predicate. A *schema (type) pattern* is a triple pattern at the schema level, ($type_1$, predicate, $type_2$), i.e., an edge of the schema graph connecting instances of $type_1$ to instances of $type_2$ via predicate. We extract these schema elements directly from the data of each KG, because neither KG ships a single authoritative schema document that matches the deployed data.

Listing 2 presents the pseudocode and queries used to extract the schema of a KG. The extraction process involves two main queries. Query 1 (line 7 in Listing 2) retrieves all types, while Query 2 (line 13 in Listing 2) extracts relationships between instances of different types. Using two optimized queries instead of a single complex query improved performance in large KGs, preventing timeout issues. For a few types, Query 2 still encountered timeouts due to the high number of instances and relationships; in such cases, simplified queries were used to mitigate the issue.

For Bio2RDF and Wikidata, the properties used in these queries differ. In Bio2RDF, `rdf:type` and `rdfs:subClassOf` are used, whereas Wikidata employs `wdt:P31` (*instance of*) and `wdt:P279` (*subclass of*) as their equivalents. Additionally, since Wikidata lacks subgraphs in its architecture, the "Graph" component is omitted from Query 2 when applied to Wikidata. For Bio2RDF we additionally keep only IRIs that match the Bio2RDF vocabulary template `http://Bio2RDF.org/DATASET_vocabulary:SCHEMA_ELEMENT` (excluding the generic `:Resource`), which yields OWL-style class and property IRIs.

~~What counts as a “class” in Wikidata — and how it is used.~~ What counts as a “class” in Wikidata, and how it is used. Bio2RDF follows a classical OWL-style design in which We distinguish two schema modeling styles that matter for measurement. In an OWL-style KG, classes and properties have are dedicated vocabulary IRIs and classes have instances, so its schema is, disjoint from the instance data, and a class is not itself a legitimate value of a property; Bio2RDF and DBpedia are of this kind, and their schemas are unambiguous to extract. ~~Wikidata is fundamentally different: its RDF model draws~~ In an item-based KG, there is no formal class/instance separation. ~~“Classes: the same kind of resource serves as both, so an entity may act as a class in one triple and as an ordinary data value in another. Wikidata is of this kind. Its “classes” are ordinary items, distinguished only by being used as the object of wdt:P31 (instance of) or wdt:P279 (subclass of); defining a Wikidata class is genuinely contested [9, 21, 24]. Crucially — and this is a measurement issue specific to item-based KGs — the~~ In the logs we analyse, the same item is far more often referenced in queries as a property value than as a class: e.g. Q33999 (actor) as the value of P106 (occupation), or Q142 (France) as the value of P27 (country of citizenship). A faithful usage analysis must therefore distinguish the position in which an element is referenced. We call an entity reference *class-position* when it is the object of wdt:P31/wdt:P279 (or of a property path involving them, such as wdt:P31/wdt:P279*), and *value-position* otherwise. Conflating the two —, as a naive “type coverage” does —, counts value usage as though it were schema usage; we therefore report the split (Section 4.8.1). This distinction does not arise for Bio2RDF, whose vocabulary classes are not used as property values. We additionally distinguish *instantiated* classes (objects of wdt:P31, ≥ 1 instance) from purely hierarchical or uninstantiated items. These distinctions are central to interpreting the Wikidata coverage numbers and to the discussion of what “schema coverage” means for an item-based KG (Section 5).

Worked example. Consider the two example queries from the Wikidata Query Service. The first, `SELECT ?cat WHERE { ?cat wdt:P31 wd:Q146 }` (house cats), references the predicate P31 and the type Q146; both are extracted as used schema elements. The second, `?horse wdt:P31/wdt:P279* wd:Q726` (horses and their subclasses), references the predicates P31 and P279 and the type Q726; these three are extracted. Note that the wdt:P279* closure means the query also retrieves subclasses of horse (e.g. *racehorse*, Q2442470), but those subclass IRIs are *not* written in the query text and are therefore not counted as explicitly referenced (we return to this distinction in Section 3.4 and Section 5).

The schema for any Bio2RDF dataset follows a specific URL template: `http://Bio2RDF.org/DATASET_vocabulary:SCHEMA_ELEMENT`. Therefore, any URLs retrieved from query execution that do not adhere to this template are excluded from the final schema elements set.

Listing 2: Pseudocode to extract KG schema. KG schema extraction is consist of two queries, Query 1 and Query 2.

```

1 for graph in the KG:
2     for each property in ['type_property', 'subclass_property']:
3         Execute Query_1 to get subject_types
4         for each subject_type and property in ['type_property', 'subclass_property']:
5             execute Query_2
6
7 Query_1:
8 SELECT DISTINCT ?subject_type
9 WHERE {
10     GRAPH <{graph}> {
11         ?s {property} ?subject_type . } }
12
13 Query_2:
14 SELECT DISTINCT subject_type ?p ?object_type
15 WHERE {
16     GRAPH <{graph}> {
17         ?s {property} {subject_type} .
18         ?s ?p ?o .
19         ?o {property} ?object_type . } }
```

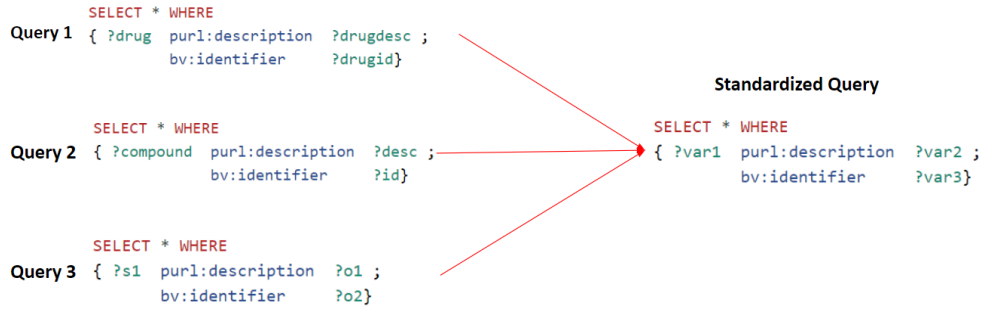


Fig. 2. Variable standardization. ?drug, ?compound, and ?s1, are standardized to ?var1.

3.3. Preprocessing SPARQL Queries

We extract unique SPARQL queries and store them along with their respective execution counts. Two queries are considered identical when their text is identical after collapsing runs of whitespace (spaces, tabs, and newlines) to a single space and trimming leading/trailing whitespace; we do not otherwise reformat the queries at this stage. We count over *unique* queries because our questions concern the *diversity* of schema elements that users and tools query, and deduplication prevents a small number of very high-frequency automated queries from dominating the distribution. We note that repetition is itself a meaningful signal of demand; deduplication is a deliberate choice to measure intent diversity rather than execution volume, and a complementary volume-weighted analysis is a natural extension. Before parsing, we preprocess the queries to ensure proper syntax and facilitate accurate parsing. First, we incorporate essential RDF syntax prefixes (such as `rdf`, `rdfs`, and `owl`) since some queries lack explicit prefix declarations. While Virtuoso pre-registers these prefixes—allowing users to omit them—our parser, ‘sparqljs’⁶, does not support this feature. Therefore, we add the necessary prefixes to all queries to ensure proper parsing.

Next, we normalize the queries for parsing. Crucially, this normalization is performed *before* deduplication, so that queries differing only in incidental HTTP artifacts collapse to a single unique query. Specifically, we (i) fully percent-decode the query text (not only `%3A`); (ii) remove the trailing chain of HTTP query-string parameters that the log capture appended to the SPARQL text (queries were submitted as `GET /sparql?query=<SPARQL>&format=...&timeout=...&callback=...`); we strip a terminal run of `&key=value` segments, anchored to the end of the string so that IRIs containing `&` mid-query are not affected; and (iii) collapse runs of whitespace. We do *not* rewrite the empty/relative IRI `<>`: doing so (e.g., to a variable) would change query semantics, so instead relative IRIs are resolved by supplying the parser a base IRI, as the Virtuoso endpoint itself does. Failing to strip the appended parameters causes many syntactically valid queries to be rejected; the dominant leaked parameter was `&callback` (from JSONP/visualization clients), ~~which earlier processing overlooked.~~

After preprocessing, we parse the queries using ~~the ‘sparqljs’~~ `sparqljs`, a SPARQL 1.1 parser implemented in JavaScript, to eliminate queries that are not syntactically valid and ~~get to obtain~~ the parse tree of ~~the queries~~ each query for content extraction. All validity counts and all used-schema-element counts reported in this paper are produced by this `sparqljs` path. Our released code additionally implements two alternative back-ends, `rdflib` (Python) and `pyoxigraph` (Rust), which we use only as an independent check that the validity rates are not an artifact of a single parser (Section 4.3); they do not contribute any reported number. A parse tree represents the syntactic structure of a query, breaking it down into its components, such as subjects, predicates, and objects. We standardize variable names, as demonstrated in Figure 2. Three equivalent queries that differ only in their variable names e.g., `?drug`, `?compound`, and `?s1`, are standardized to `?var1`. We distinguish variables from actual entities by substituting variable names with placeholders like ‘varX’. The same preprocessing and variable standardization are applied identically to both the Bio2RDF and Wikidata logs. The output of this preprocessing is unique, normalized, valid queries.

⁶<https://www.npmjs.com/package/sparqljs>

3.4. Extracting Used Schema Elements

We write a Python script to extract “used schema elements” from the parse trees of the queries. We first identify the subject, predicate, and object IRIs within each query’s parse tree. Then, we determine the intersection of these IRIs with the total schema elements that were retrieved earlier.

What we mean by “used”. Throughout, a schema element is counted as *used* if it is *explicitly referenced*, i.e., its IRI is written in the query text—the *syntactic-explicit* usage notion introduced in Section 1. We deliberately adopt this explicit-reference notion rather than a closure- or answer-based (*semantic*, effective) one. As illustrated by the horse example in Section 3.2, a query containing `wdt:P31/wdt:P279* wd:Q726` explicitly references only `P31`, `P279`, and `Q726`; the subclasses of horse that the property path traverses at evaluation time (e.g. *racehorse*) contribute to the *answers* but are not named in the query, and a similar effect arises with property hierarchies. Explicit reference is the appropriate notion for this paper’s motivations: documentation, schema and interface design, autocomplete, and deprecation decisions all act on the elements that users and tools actually *write*, not on the transitive closure a query happens to traverse. We quantified how often the two notions could diverge by measuring property-path usage. Property paths of any kind appear in only about 1% of valid Bio2RDF organic queries (98 of 8,140), but in 21.1% of Wikidata organic queries in 2017 (18,705 of 88,491) and 13.7% in 2018 (25,351 of 185,443); restricting to the transitively-traversing `*/+` operators—the ones that could actually pull in additional schema elements—the 2017 figure is 16.6% (14,696 of 88,491). The explicit-reference and closure-based notions therefore coincide for nearly all Bio2RDF queries, whereas for Wikidata a non-trivial fraction of queries traverse the subclass hierarchy that a closure-based notion would additionally credit.

We can bound this divergence directly. In the 2017 organic queries of Wikidata, the subclass-traversing paths (`wdt:P279*`, `wdt:P31/wdt:P279*`) are anchored at 1,276 distinct classes; the transitive subclass closure of those anchors, computed from the 2017 dump’s 2.08M `P279` edges, spans 64,843 of the KG’s $\sim 103,000$ classes, of which 62,410 are not named explicitly. A closure/answer-based “effective usage” notion would therefore credit up to those 62,410 additional types, raising organic coverage from $\sim 4\%$ to $\sim 62\%$ —a +60 percentage-point swing. Crucially, this bound is dominated by a handful of queries anchored at near-root classes: `wdt:P279*` from *entity* (`Q35120`) alone, used in just 13 queries, has a closure of 64,356 classes, and *concept* (`Q151885`) 49,828, whereas typical anchors are small (*city* 96, *film* 94, *organization* 3,538). A closure-based metric is thus not merely harder to compute but ill-behaved on an item-based KG: a single query rooted at *entity* would mark most of the ontology as “used,” conflating *queried* with *reachable in principle*. This is positive evidence for the explicit-reference choice; a principled, partially-credited closure measure remains future work (Section 5).

3.5. Calculating Schema Coverage

Equation 1 was used to calculate the SPARQL Schema Coverage (SC). This metric quantifies the proportion of the KG schema that is used by user queries. SC is the ratio of Used Schema Elements (USE) to Total Schema Elements (TSE) in the KG, multiplied by 100 to express the result as a percentage. The TSE represents the count of all distinct types and predicates in the KG, while the USE denotes the count of those classes and predicates found in user SPARQL query logs. We obtained the values for USE and TSE from the results of the previously described steps. Coverage is always reported against a *clearly delimited reference set*: the numerator and denominator are derived from the same set of subgraphs (Section 4.1), and we report types and predicates separately (Table 4) rather than only their sum. Because the type sets of Bio2RDF and Wikidata are constructed differently (OWL-style vocabulary IRIs vs. the data-driven `wdt:P31/wdt:P279` operationalization of Section 3.2) and differ in scale by roughly two orders of magnitude, coverage percentages are meaningful *within* a KG (across query types and time) but should not be read as a direct cross-KG comparison.

$$SC(\%) = \left(\frac{USE}{TSE} \right) \times 100 \quad (1)$$

3.6. Identifying Content Usage Patterns

Up to this point, we have introduced a method for calculating SPARQL schema coverage, which measures the proportion of schema elements in a KG that are queried. However, this metric alone does not reveal how frequently individual schema elements are used or whether their usage is consistent over time. Step six involves analyzing the frequency distribution of the schema elements, identifying frequently used elements, and detecting commonalities in robotic and organic logs, or across different time periods. Additionally, we apply statistical tests, including the Wilcoxon signed-rank test and Spearman's rank correlation coefficient, to assess the consistency and changes in the frequency and ranking of schema element usage over time. The following sections provide a detailed explanation of these methods, with the findings presented in the Results section.

3.6.1. Frequency Distribution of Schema Elements

To analyze and compare the frequency distributions of schema elements across multiple datasets, we normalize the usage count of each element to account for varying time intervals. We calculate the monthly frequency of each element by dividing its total count by the log collection period expressed in months (Equation 2). Here one month is a fixed 30-day period, so the normalized count has units of *uses per 30-day month* (it is not dimensionless). To reduce skewness and enhance interpretability, we apply a logarithmic transformation ($\log_{10}$) to these normalized counts. Finally, we visualize the normalized frequency distribution of schema elements across different datasets to facilitate comparison.

$$\text{Normalized count} = \frac{\text{Total count}}{\log \text{ collection period}} \quad (2)$$

Before comparing the two intervals statistically, we must decide whether a parametric (normality-assuming) test is appropriate. The Shapiro–Wilk test (Equation 3) assesses whether a sample plausibly comes from a normal distribution; we use it purely as a precondition check. We apply it to the per-element *change* in normalized usage frequency between the two intervals (defined below). We emphasize that normality here is a property of these numeric differences (real numbers, naturally ordered), not of any ordering over the schema elements themselves.

$$SW = \frac{(\sum_{i=1}^n a_i x_i)^2}{\sum_{i=1}^n (x_i - \bar{x})^2} \quad (3)$$

where the x_i are the ordered sample values, $\bar{x}$ is the sample mean, and the a_i are the Shapiro–Wilk coefficients, i.e., tabulated constants derived from the expected values and covariances of the order statistics of a standard-normal sample of size n . In our case each x_i is, for one schema element common to both intervals, the change in its normalized monthly usage frequency from the first interval to the second (interval-2 value minus interval-1 value).

The test returned $p \approx 0$ for both the Wikidata and Bio2RDF interval pairs, so the per-element changes are not normally distributed. We therefore use the non-parametric Wilcoxon signed-rank test (Equation 4), which compares the same schema element measured in two intervals (a paired design) without assuming normality, to test whether usage frequencies shifted significantly between the two intervals.

$$W = \sum_{i=1}^n \text{sign}(d_i) R_i \quad (4)$$

where d_i is the difference between the usage frequency of schema elements (in two intervals), $\text{sign}(d_i)$ is the sign of d_i , and R_i is the rank of the absolute differences.

Changes in Used Schema Element Ranking. The Wilcoxon test addresses changes in usage *magnitude*; we separately ask whether the relative *popularity ranking* of schema elements is stable over time. We rank elements by their normalized usage frequency within each interval (rank 1 = most frequently used) and compute Spearman's rank correlation coefficient (ρ) (Equation 5) between the two rankings. Spearman's ρ is the Pearson correlation of the ranks; it summarizes in a single number in $[-1, 1]$ whether elements that were popular in one interval remained popular in the other, and it is distribution-free, which suits our non-normal data. Here d_i is the difference between the ranks of schema element i in the two intervals, and n is the number of common schema elements [28]. We use Spearman's ρ as a compact measure of temporal *stability*, complementing the Wilcoxon test.

$$\rho = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)} \quad (5)$$

3.6.2. Frequent Schema Elements

We analyze the most frequently queried schema types from robotic and organic logs of Wikidata and Bio2RDF to identify common elements that persist across different time intervals and log types. This focus on the most frequent elements is similar in spirit to the predicate-frequency analysis of Bonifati et al. [7], to which we add the temporal and organic/robotic dimensions. While our ranking analysis of all schema elements reveals broad usage dynamics and temporal shifts, we're particularly interested in determining whether a subset of frequently accessed elements remains consistently popular over time, indicating their significance as "core" elements of the KG. The presence of a stable core across the datasets would suggest that these elements are central to the KG's utility, regardless of the query time or type. This frequent-element analysis is computed over schema *types* (classes); predicates, whose frequency distribution is dominated by a few generic relations, are analyzed separately. To investigate the stable core, we first examine the top 10 schema types in each dataset, as they represent the most frequently queried components, as well as the top 50 types. The top 50 threshold strikes a balance by providing a more comprehensive view of usage patterns compared to the top 10 while still avoiding the inclusion of many low-frequency types. For example, in Wikidata 2017, the frequency for types ranked 51 to 3560 ranges from 149 to 1, whereas the top 50 types have frequencies between 6,962 and 150. To analyze schema usage patterns, we visualize frequently queried schema elements and identify common elements across datasets, log types, and time intervals. This approach aims to determine whether there is a set of core schema elements that drives query activity in these KGs.

Pairwise Schema Type Usage of Bio2RDF. Bio2RDF consists of 26 interconnected subgraphs, each representing a distinct dataset with its own vocabulary. These subgraphs are linked through various predicates. For example, the `ctd_vocabulary:Gene-Disease-Association` type from the "ctd" subgraph and the `ncbigene_vocabulary:Resource` from the "ncbigene" subgraph are linked by the predicate `ctd_vocabulary:gene`, establishing a relationship between the two elements. A typical triple pattern representing the connection between two distinct subgraphs is: `?schema_Type_from_first_subgraph ?predicate ?schema_Type_from_second_subgraph`. This relationship is referred to as a "pairwise schema type pattern."

Bio2RDF's subgraph organization enables an analysis of how frequently queries capture relationships between schema types from different subgraphs. However, since Wikidata does not have an explicit subgraph structure, this type of analysis is not applicable to its dataset. To investigate this type of usage, we conduct an experiment comparing Bio2RDF query logs from 2013 and 2019 to determine how often datasets (or subgraphs) were integrated in queries to retrieve more complex information. The approach consists of two main steps:

Identifying total pairwise schema type patterns. We extract all possible pairwise schema type patterns from the Bio2RDF schema, covering 17 subgraphs in `log_2013` and 26 subgraphs in `log_2019`.

Identifying pairwise schema type pattern occurrences in logs. We analyze the triple patterns within queries to identify occurrences of these pairwise schema type patterns. For each dataset, we construct a matrix where both rows and columns represent schema type subgraphs. Each cell in the matrix corresponds to a specific pair of subgraphs, and its value indicates the frequency of the pattern in the queries. By conducting this analysis, we aim to discover how often queries integrate information across Bio2RDF subgraphs and how this usage has evolved over time.

4. Results

In this section, we describe the results of the various steps in the method, including the retrieval of datasets, the pre-processing of SPARQL queries, the extraction of all schema elements, the extraction of the schema elements used, the calculation of the SPARQL schema coverage, and the analysis of usage patterns.

4.1. Dataset Retrieval

Table 1 provides an overview of the datasets retrieved for analysis, including the logs from Bio2RDF and Wikidata KGs for multiple periods and query types. We use a single naming convention of the form *KG-agenttype-period[-interval](KG-version)*; for example, *Bio2RDF-all-2013* is the full 2013 Bio2RDF log and *Wikidata-organic-2017-int1* is interval 1 of the 2017 Wikidata organic log. Because most analyses evaluate a log against a specific KG version, we extend the same convention with the version it is paired with, written *log(period)_kg(version)*: for instance *Bio2RDF-all-log2013_kg2024* denotes the *Bio2RDF-all-2013* log evaluated against the 2024 Bio2RDF schema. Tables 1–5 and the figure panels use these two forms throughout, the plain form where only the log matters and the paired form where the KG version does. The *Bio2RDF-all-2013* log originates from LSQ and spans 16.79 months, from May 2013 to September 2014, containing 31,950,877 queries. This LSQ-derived dataset lacks user-agent information and only includes the `lsqv:hostHash` property, an anonymized hash of the host or IP address issuing each query; the LSQ release therefore cannot itself be split into organic and robotic. To recover that split for the 2013 era we additionally obtained the *raw* Bio2RDF server access log (the upstream source from which LSQ was built, spanning May 2013–September 2015), which retains user agents; we use it to characterize 2013-era organic vs. robotic traffic in Section 4.7.1. The second dataset, *Bio2RDF-all-2019*, covers a longer period of 29.93 months, from January 2019 to July 2021. It comprises 3,880,960 log entries. Of these, 3,212 are server-side log lines that contain no SPARQL query, leaving 3,877,748 SPARQL queries. This dataset retains user-agent information, allowing us to separate robotic (3,816,776) and organic (56,736) queries; a further 7,427 queries lack user-agent information and are excluded from the split.

Wikidata logs contain both organic and robotic queries. The *Wikidata-organic-2017-full* log spans 9.43 months, from June 2017 to March 2018, and consists of 3,298,254 organic queries. The shorter, interval-based slices used for the temporal comparison were collected over only 0.88 months each: *Wikidata-organic-2017-int1* and *Wikidata-organic-2018-int7* comprise 192,331 and 872,556 organic queries, respectively, while *Wikidata-robotic-2017-int1* and *Wikidata-robotic-2018-int7* contain 59,355,579 and 81,339,186 robotic queries, respectively.

Table 1

Results of the data retrieval step. Datasets follow the naming convention *KG-agenttype-period[-interval]*; “All” denotes the union of organic and robotic queries (not a superset of any single row).

Dataset	#Queries	log collection period	Duration (months)
Bio2RDF All log2013	31,950,877	2013-05-05 to 2014-09-28	16.79
Bio2RDF All log2019	3,880,960	2019-01-07 to 2021-07-06	29.93
Bio2RDF robotic log2019	3,816,776	2019-01-07 to 2021-07-06	29.93
Bio2RDF organic log2019	56,736	2019-01-07 to 2021-07-06	29.93
Wikidata All organic log2017	3,298,254	2017-06-11 to 2018-03-25	9.43
Wikidata robotic log2017	59,355,579	2017-06-12 to 2017-07-09	0.88
Wikidata robotic log2018	81,339,186	2018-02-26 to 2018-03-25	0.88
Wikidata organic log2017	192,331	2017-06-12 to 2017-07-09	0.88
Wikidata organic log2018	872,556	2018-02-26 to 2018-03-25	0.88

During our analysis in 2024, the Bio2RDF SPARQL endpoint contained 26 subgraphs/datasets. However, the Bio2RDF log2013 and Bio2RDF log2019 datasets do not contain query logs corresponding to all 26 subgraphs. Figures 3 and 4 provide details through Venn diagrams: Figure 3 illustrates the overlap between the Bio2RDF log2013 datasets and the KG subgraphs, while Figure 4 represents the Bio2RDF log2019 datasets compared to the KG subgraphs in the 2024 endpoint. This comparison highlights the importance of using a consistent reference

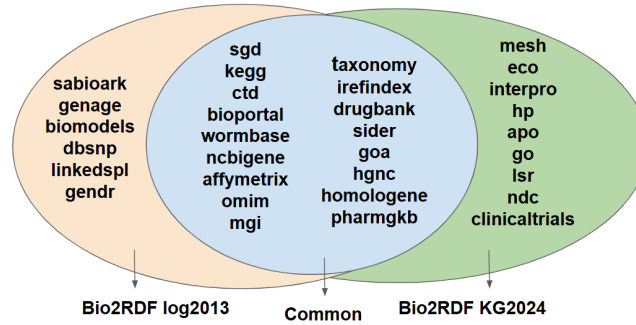


Fig. 3. Venn diagram illustrating the overlap between the Bio2RDF log2013 datasets and the Bio2RDF KG 2024

set when calculating schema coverage (Equation 1). That is, the numerator and denominator (i.e., used and total schema elements) should be derived from the same set of datasets, ensuring alignment between KG subgraphs and log datasets.

The Venn diagram in Figure 3 compares the datasets included in Bio2RDF log2013 with the subgraphs in Bio2RDF KG2024. The left oval represents datasets unique to Bio2RDF log2013. The overlapping section in the center lists 17 datasets common to both Bio2RDF log2013 and Bio2RDF KG2024. The right oval represents datasets unique to Bio2RDF KG2024.

The Venn diagram in Figure 4 compares the datasets included in Bio2RDF log2019 with the subgraphs in Bio2RDF KG2024. The left oval represents datasets unique to Bio2RDF log2019. The overlapping section in the center lists 22 datasets common to both Bio2RDF log2019 and Bio2RDF KG2024. The right oval represents datasets unique to Bio2RDF KG2024.

The dataset "Bio2RDF" in log2019 refers to the logs of a general endpoint for all subgraphs and may include queries to the HP, ECO, APO, and GO subgraphs. This is because, historically, Bio2RDF maintained a distinct SPARQL endpoint for each subgraph. Later, a single general endpoint was introduced to host all subgraphs for querying.

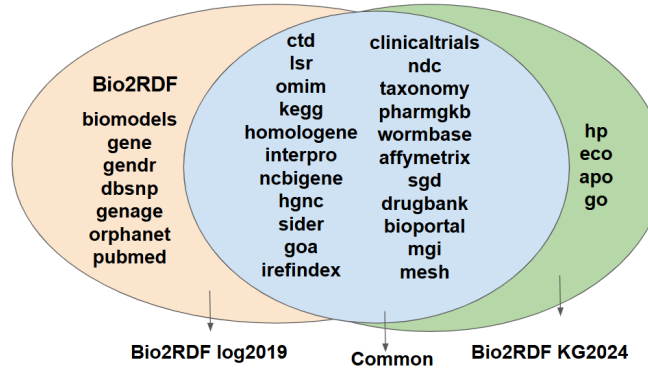


Fig. 4. Venn diagram illustrating the overlap between the Bio2RDF log2019 datasets and the Bio2RDF KG 2024

4.2. Extracting Total Schema Elements

Table 2 presents the statistics of total schema elements across different KG versions. Initially, the Bio2RDF schema comprised 166,444 unique types. After excluding types that do not adhere to the template `http://Bio2RDF.org/DATASET_vocabulary:SCHEMA_ELEMENT`, 545 types remain. A further refinement, by excluding elements with the template `http://Bio2RDF.org/DATASET_vocabulary:Resource`, reduces the count to 350 types and 195 predicates, distributed across 26 subgraphs within the Bio2RDF KG on the current endpoint (2024). Out of

these 26 subgraphs, only 17 have corresponding query logs available in the Bio2RDF log2013 dataset, as depicted in Figure 3. These 17 subgraphs contain 270 types and 129 predicates, in total 399 schema elements.

As represented in Table 2, the Wikidata KG version 2017 contains 103,380 types, whereas the Wikidata KG version 2018 contains 97,470 types, reflecting a reduction in the number of schema types. These results illustrate the variability in schema complexity with Wikidata exhibiting a larger and more diverse set of schema elements compared to Bio2RDF.

Table 2
Total schema elements Statistics per KG version

Source	Bio2RDF KG2024 (17 subgraphs)	Bio2RDF KG2024 (26 subgraphs)	Wikidata KG2017	Wikidata KG2018
Total number of distinct Elements	399	545	104,314	98,462
Number of distinct Types	270	350	103,380	97,470
Number of distinct Predicates	129	195	934	992

4.3. Preprocessing SPARQL Queries

Table 3 summarizes the number of queries (#Queries), unique queries (#Unique), queries with incorrect syntax (#Failed to parse), and unique syntactically valid queries (#Valid) across all datasets. ~~We emphasize the Bio2RDF 2019 rows : with the corrected preprocessing, the number of valid queries is far higher than under the earlier pipeline (e.g., for robotic 2019, differ most from the preliminary work [20]; for Bio2RDF-all-2019, the dataset also used there, the corrected parameter stripping yields 1,034,040,831 vs. 627,666 valid queries against 634,314), because previously 783 previously, because the appended HTTP query-string parameters had caused a large share of well-formed queries to be discarded. The LSQ-sourced Bio2RDF 2013 and the Dresden-sourced Wikidata logs were cleanly extracted and reproduce closely (validity rates of 99.8% and >99.9% respectively), confirming that the parsing defect was specific to the raw Bio2RDF 2019 server logs and does not affect the schema-coverage results, which are computed over the same valid queries~~

~~The variable standardization of Section 3.3 is the last preprocessing step, and it is worth recording how much it actually merges, since two queries that differ only in variable naming should not count as distinct usage. Its effect is small in every log we measured: applying it to the valid queries reduces Bio2RDF-organic-2019 from 8,208 to 8,099 distinct query shapes (1.3%), Bio2RDF-robotic-2019 from 1,034,833 to 1,034,754 (0.01%), Wikidata-organic-2017-int1 from 88,514 to 88,116 (0.45%), Wikidata-organic-2018-int7 from 185,478 to 184,934 (0.29%), and the multi-million-query Wikidata-all-2017-int1 from 8,317,396 to 8,316,939 (0.01%). Variable naming is therefore almost never the only difference between two logged queries, so standardization matters for correctly extracting schema elements from the parse tree rather than for deduplication.~~

~~**What the invalid queries are, and whether validity is parser-dependent.** Three separate observations about the Bio2RDF 2019 logs are worth recording here, since they bear on how the validity counts in Table 3 should be read.~~

~~First, the two query types differ sharply in syntactic hygiene. About 99.5% of unique robotic queries parse, whereas a large fraction of unique organic entries are not SPARQL at all. Inspecting them shows injected non-SPARQL text, mostly blog-comment spam submitted to the public endpoint form (one entry reads “I love your blog... very nice colors & theme”), together with nine entries consisting only of the string /sparql. This is a property of an open web form on a public endpoint, not a property of human querying as such.~~

~~Second, among the queries that are non-standard rather than non-SPARQL, the Virtuoso extensions `bif:contains` (full-text search), `OPTION`, and the `LIKE` operator occur overwhelmingly in organic rather than robotic traffic. We report this as a property of these Bio2RDF logs and do not generalise it: the organic stream here reaches the endpoint through a web form that exposes such syntax, so the contrast may reflect the access path rather than any inherent difference between human and automated clients.~~

~~Third, the validity rate is not an artifact of the parser we chose. We re-validated the same queries with three independent SPARQL 1.1 implementations, a strict grammar, the `sparqljs` parser used throughout, and a Rust engine, which agree on validity for 98–99% of queries.~~

Table 3

Results of parsing and validating the query logs, regenerated with the corrected preprocessing (full HTTP query-string parameter stripping applied before deduplication, base-IRI resolution of relative IRIs). #Failed-to-parse = #Unique – #Valid. #Valid is computed with [the sparqljs](#) SPARQL 1.1 parser([sparqljs](#)). The Bio2RDF 2019 rows change substantially relative to earlier processing because appended HTTP parameters (notably `callback`) had previously caused many valid queries to be rejected; the LSQ-sourced (Bio2RDF-2013) and Dresden-sourced (Wikidata) logs were already cleanly extracted and reproduce. The Bio2RDF 2013 figures use the executed-query set (queries with a recorded remote execution).

Dataset	#SPARQL Queries	#Unique	#Failed to parse	#Valid
Bio2RDF All log2013	31,950,877	1,519,681	2,521	1,517,160
Bio2RDF All log2019	3,880,939	1,050,616	9,950	1,040,666
Bio2RDF robotic log2019	3,824,203	1,040,356	5,525	1,034,831
Bio2RDF organic log2019	56,736	12,579	4,439	8,140
Wikidata All organic log2017	3,530,948	859,290	321	858,969
Wikidata robotic log2017	59,355,579	8,234,089	20	8,234,069
Wikidata robotic log2018	81,339,186	18,939,743	86	18,939,657
Wikidata organic log2017	192,330	88,514	23	88,491
Wikidata organic log2018	872,555	185,478	35	185,443

4.4. Extracting Used Schema Elements

Table 4 presents the statistics of used schema elements in various datasets, detailing the total number of distinct elements, types, and predicates identified in the queries. From Bio2RDF log2019_kg2024, 529 distinct elements were extracted, comprising 334 distinct types and 195 distinct predicates. In comparison, Bio2RDF log2013_kg2024 contains 317 distinct elements, with 227 distinct types and 90 predicates. The robotic queries for Bio2RDF log2019 match the overall log2019_kg2024 dataset, both reporting 529 distinct elements, 334 classes, and 195 predicates, while the organic queries yield a much smaller subset of 116 elements, 58 types, and 58 predicates.

For Wikidata, log2017_kg2017 includes 13,394 distinct elements, 12,590 classes and 804 predicates, while log2017_kg2018 has 13,900 distinct elements, consisting of 13,054 types and 846 predicates. The robotic queries for Wikidata scale significantly larger, with robotic log2017_kg2017 containing 59,603 elements, 58,680 types, and 923 predicates, and robotic log2018_kg2018 reporting 59,582 elements, 58,648 types, and 934 predicates. These results illustrate significant variability in schema usage between robotic and organic logs.

Table 4

Used schema-element statistics. Each row is based on a *pair* of datasets: a query log and the KG version it is evaluated against, written *log_kg* (e.g. *Bio2RDF-all-log2013_kg2024* = the 2013 log matched against the 2024 KG). Counts are over unique normalized queries.

Dataset	Total number of distinct Elements	Number of distinct Types	Number of distinct Predicates
Bio2RDF All log2013_kg2024	317	227	90
Bio2RDF All log2019_kg2024	529	334	195
Bio2RDF robotic log2019_kg2024	529	334	195
Bio2RDF organic log2019_kg2024	116	58	58
Wikidata All organic log2017_kg2017	13,394	12,590	804
Wikidata All organic log2017_kg2018	13,900	13,054	846
Wikidata robotic log2017_kg2017	59,603	58,680	923
Wikidata robotic log2018_kg2018	59,582	58,648	934
Wikidata organic log2017_kg2017	4,041	3,559	482
Wikidata organic log2018_kg2018	4,859	4,310	549

4.5. Calculating SPARQL Schema Coverage

Table 5 presents the SPARQL schema coverage of datasets and provides insight into the extent to which schema elements of different KGs are used in SPARQL queries. Bio2RDF All log2019_kg2024 and Bio2RDF robotic

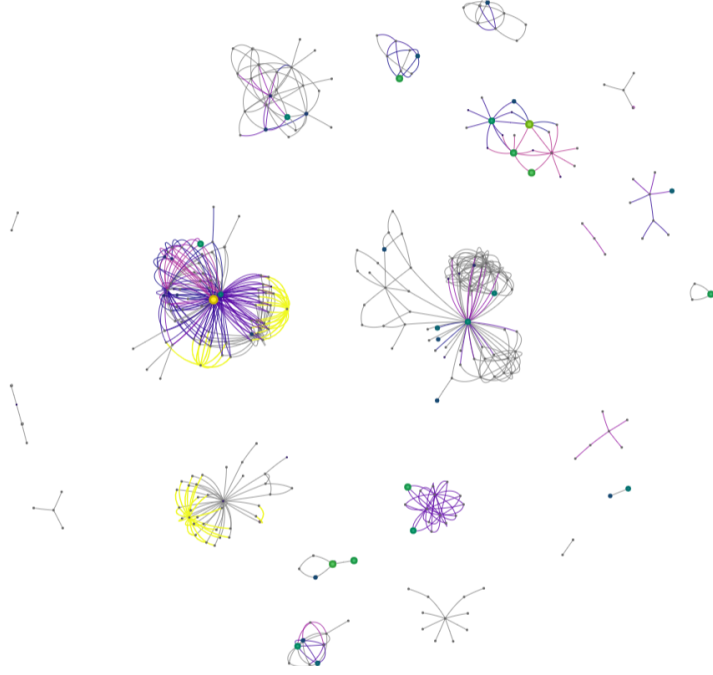


Fig. 5. Visualization of schema-element usage in *Bio2RDF-organic-2019* (KG2024). Node size and color both increase with usage frequency (small/dark = rarely used, large/bright = heavily used); edge width increases with predicate usage; schema elements not used in any query are gray. The few large, bright hubs concentrate most of the usage.

log2019_kg2024 achieved the highest schema coverage at 97.06%, indicating that nearly all schema elements were queried at least once. This is followed by Bio2RDF All log2013_kg2024, which achieved a schema coverage of 79.44%.

In contrast, the Bio2RDF Organic log2019_kg2024 exhibited a much lower schema coverage of 21.28%. Similarly, the schema coverage for Wikidata robotic queries in log2017_kg2017 and log2018_kg2018 was moderate, at 57.13% and 60.51%, respectively. However, the schema coverage for Wikidata organic queries in log2017_kg2017 and log2018_kg2018 was significantly lower, at 12.84% and 14.11%, respectively.

Table 5

SPARQL schema coverage. Each row pairs a query log with a KG version (*log_kg*); coverage is $USE/TSE \times 100$ over the same reference set of subgraphs. As discussed in Sections 3.5 and 4.6, these percentages are comparable within a KG but not directly across the two KGs.

Dataset	SC
Bio2RDF All log2013_kg2024	79.44%
Bio2RDF All log2019_kg2024	97.06%
Bio2RDF robotic log2019_kg2024	97.06%
Bio2RDF organic log2019_kg2024	21.28%
Wikidata All organic log2017_kg2017	12.84%
Wikidata All organic log2017_kg2018	14.11%
Wikidata robotic log2017_kg2017	57.13%
Wikidata robotic log2018_kg2018	60.51%
Wikidata organic log2017_kg2017	3.87%
Wikidata organic log2018_kg2018	4.93%

These results highlight a notable difference in schema coverage between robotic and organic queries across Bio2RDF and Wikidata datasets.

Figure 5 illustrates the extent to which schema elements of the Bio2RDF KG are used in the organic queries of 2019. Nodes represent schema types and edges represent predicates linking instances of those types. Both the size and the color of a node encode the same quantity, its usage frequency: nodes grow larger and brighter as they are queried more often, while rarely used nodes are small and dark. Edge width likewise grows with predicate usage, and schema elements that appear in no query are shown in gray. The visualization makes the long tail visible directly: a handful of large, bright hubs (drug, gene, and disease types) concentrate the bulk of organic usage, while most of the schema remains gray and unqueried. The interactive version, with exact per-element values, is available online⁷.

What the Bio2RDF organic-2019 log actually contains. The organic/robotic split (Section 2) is a user-agent proxy, and a browser user-agent does not guarantee a human information need. Inspecting the 8,140 valid unique organic-2019 queries shows that **86.9% reference no Bio2RDF schema element at all**: they are the endpoint's default-sample query (`select * where {a ?Concept} LIMIT 100`), liveness checks (`select ?s where {?s ?p ?o} limit 1`), Virtuoso-internal test queries with `OPTION (RVR/BREAKUP)` against no-such-IRIs, and bare label/dereference lookups — automated or non-informative browser traffic rather than human information-seeking, the 2019 counterpart of the infrastructure contamination we document for the raw 2013 log (Section 4.7.1, Table 8). Only **1,065 queries (13.1%) are schema-bearing**, and it is these that produce the 116 used elements and the 21.3% organic coverage. This does not inflate coverage or concentration — schema-less queries contribute *zero* used elements, so the numerator is drawn entirely from the schema-bearing minority — but it does mean the Bio2RDF organic-2019 figures characterise a small core of genuine human queries, not the raw browser stream, and any organic/robotic behavioural contrast on Bio2RDF-2019 should be read with that in mind. We therefore restrict the query-template analysis (Section 4.9.7) to schema-bearing queries.

4.6. Size-Controlled Coverage: Is the Organic/Robotic Gap a Sampling Artifact?

The coverage values in Table 5 could be misleading, because organic and robotic logs differ enormously in size (Table 3), and **larger samples mechanically touch more vocabulary**. To separate a genuine behavioral difference from this sampling effect, a larger sample of schema-element occurrences cannot recover fewer distinct elements than a smaller sub-sample drawn from the same log. To assess whether the observed coverage difference persists after controlling for sampling effort, we treat each occurrence of a schema element in a log as an “individual” and apply individual-based rarefaction [15]: **for the**, One occurrence is one reference to one schema element by one unique query, with the SPARQL `a` keyword counted as `rdf:type`, so a log's total effort is the sum over schema elements of the number of unique queries referencing each: 4,855 for Bio2RDF-organic-2019 and 1,279,731 for the Wikidata organic log. The same element referenced twice within one query is counted once, and repeated executions of an identical query are not counted again: **for the** larger (robotic) log we compute the *expected* number of distinct schema elements that would be recovered from a random sub-sample **of the containing the same number of occurrences size** as the organic log. Throughout, *sampling effort* means this number of observed schema-element occurrences, not the number of queries. This monotonicity is a property of sub-sampling *within* a single log; it does not imply that a larger log must show higher coverage than a different, smaller one. Indeed Section 4.9.4 shows that organic coverage saturates well below the volumes we observe, so a $4.7\times$ range in query volume across windows leaves coverage nearly unchanged. As a complementary, **sampling-size-independent check** estimate of total schema richness, we also computed the Chao1 richness estimator [11]; **it points the same way**, which leads to the same conclusion: for Bio2RDF the organic estimate, ≈ 144 , sits only modestly above the 116 observed, indicating little hidden richness, whereas the robotic estimate equals its observed **529**. Figure 6 shows the resulting rarefaction curves and Table 6 the equal-effort comparison. **One caveat: Two caveats apply.** First, individual-based rarefaction assumes element occurrences are exchangeable draws, whereas query traffic is bursty and correlated (a client repeats similar queries); computing over deduplicated unique queries dampens this, but the assumption is not fully met, so the equal-effort figures are best read as estimates. Second, the effort totals are not composed of equally informative draws: **for Bio2RDF-organic-2019**, 3,603 of the 4,855 occurrences (74%) are the single generic predicate `rdf:type`, which every type-assertion query names. Because the robotic log is rarefied to that same total, and `rdf:type` is similarly

⁷<https://marmhm.github.io/Schema-usage-graph/>

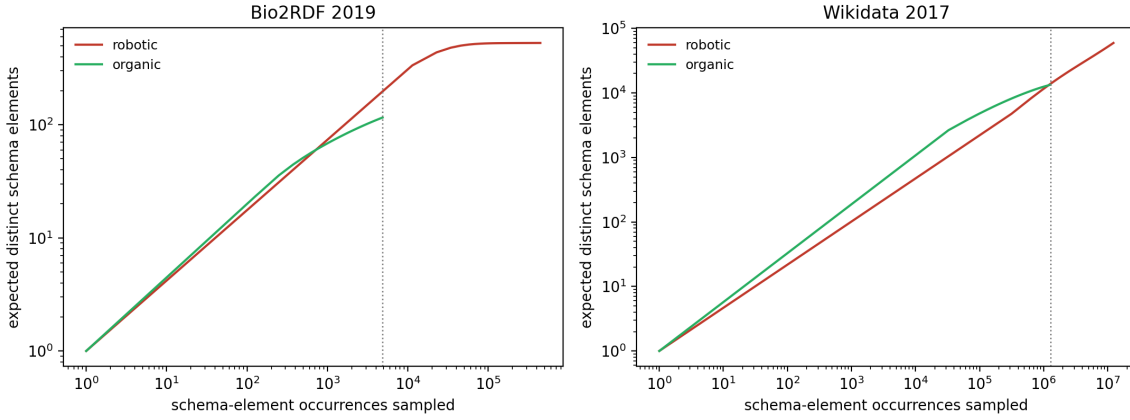


Fig. 6. Individual-based rarefaction curves (expected distinct schema elements vs. schema-element occurrences sampled, both axes log-scaled) for organic (green) and robotic (red) logs. The dotted line marks the organic log’s sampling effort. For Bio2RDF the robotic curve rises well above organic at equal effort; for Wikidata the two curves are close, and organic is in fact richer per occurrence until effort equalizes.

ubiquitous there, this affects both sides in the same direction, but it does mean the nominal effort overstates how much distinguishing evidence either sub-sample carries. The Bio2RDF coverage gap ($1.79\times$) is large enough to survive plausible deviations; the Wikidata near-parity ($1.03\times$) is where the assumption matters most, and there the conclusion (no meaningful coverage gap) is the conservative one.

Table 6

Schema coverage at equal sampling effort. Robotic logs are rarefied down to the number of schema-element occurrences observed in the corresponding organic log.

Comparison (equal effort)	Organic distinct elements (coverage)	Robotic distinct elements, rarefied (coverage)
Bio2RDF 2019 ($n = 4,855$ occ.)	116 (21.3%)	≈ 208 (38.2%)
Wikidata 2017 ($n = 1,279,731$ occ.)	13,394 (12.8%)	$\approx 13,846$ (13.3%)

The two KGs behave **oppositely differently, in opposite directions**. For **Bio2RDF**, even at identical sampling effort the robotic log recovers roughly $1.8\times$ more distinct schema elements than the organic log (208 vs. 116; 38.2% vs. 21.3%), and its rarefaction curve saturates near the full schema. A Monte-Carlo subsampling bootstrap (400 replicates, robotic rarefied to the organic effort of 4,855 occurrences) puts this equal-effort ratio at **$1.79\times$, 95% CI [1.62, 1.96]**, comfortably above 1, so robotic clients genuinely exercise a broader part of this small, specialized schema, consistent with automated enumeration or dumping. For **Wikidata**, the picture reverses: at equal effort the same bootstrap gives a ratio of only **$1.03\times$, 95% CI [1.02, 1.05]** (robotic 13.3% vs. organic 12.8% of TSE), and at *lower* effort organic queries are actually richer per occurrence (Figure 6, right). The raw coverage gap in Table 5 (57% vs. 13%, i.e. $4.4\times$) therefore collapses to a $\sim 3\%$ difference once effort is equalized: **it is almost entirely an artifact of, which is consistent with that gap being largely attributable to** robotic logs being two orders of magnitude larger **not evidence of rather than to** broader robotic interest. (With $n = 1.28\text{M}$ the tiny residual is statistically distinguishable from 1.0, but its magnitude $\rightarrow 3\%$ vs. Bio2RDF’s 79% $\rightarrow$ is negligible.)

This size dependence is reinforced by how concentrated each log is on once-seen elements (Table 7). Two thirds of the schema elements counted for *Wikidata-robotic-2017* are used exactly once; removing these singletons (the elements most exposed to one-off or erroneous uses) drops its coverage from 57.1% to 18.9% and shrinks the robotic-to-organic coverage ratio from $4.4\times$ to $2.0\times$. By contrast, *Bio2RDF-robotic-2019* has a single singleton: its breadth is exercised repeatedly and is robust. In practical terms, **for the two logs analysed here**, robotic “coverage” on **a very large KG such as** Wikidata largely reflects query volume and a long tail of one-off references, whereas on **a compact KG such as** Bio2RDF it reflects systematic, repeated traversal of the schema. **We state this as a contrast between these two datasets rather than between large and compact KGs in general: with two KGs we cannot separate the effect of schema size from other differences in their data, user communities, and query workloads.**

Table 7

Singleton sensitivity. Schema elements used exactly once, and coverage with vs. without singletons (2017 logs for Wikidata, 2019 for Bio2RDF).

Log	Used elements	Singletons (%)	Coverage	Coverage w/o singletons
Wikidata robotic	59,603	39,900 (67%)	57.1%	18.9%
Wikidata organic (full)	13,394	3,412 (25%)	12.8%	9.6%
Bio2RDF robotic	529	1 (0.2%)	97.1%	96.9%
Bio2RDF organic	116	32 (28%)	21.3%	15.4%

Validity of the extracted classes: Because a Wikidata “class” is just an item used as the object of `wdt:P31`

4.7. Bio2RDF-specific Analyses

Having established the general coverage metric (Section 4.5) and the equal-effort comparison (Section 4.6), we now group the analyses that apply only to Bio2RDF: what its organic log actually contains, the recovery of an `organic/wdt:P279`, one must ask how many extracted classes are genuine schema rather than uninstantiated or one-off (possibly erroneous) edits. We characterize each class by its *support* — the number of times it is the object of `wdt:P31` or `wdt:P279` in the KG (instances plus subclasses). Of the 103 robotic split for the 2013 era, and the effect of matching the schema to the release each log actually queried.

What the Bio2RDF organic-2019 log actually contains. The organic/robotic split (Section 2) is a user-agent proxy, and a browser user agent does not guarantee a human information need. Inspecting the 8,355 extracted 2017 classes, 42.3% are instantiated (≥ 1 instance), 88.2% are embedded in the subclass hierarchy, and 30.3% are *singletons* (used as a class exactly once) — the elements most exposed to data glitches. Crucially, *querying selects for well-supported classes*: among the classes referenced in organic-2017 queries 140 valid unique organic-2019 queries shows that **86.9% reference no Bio2RDF schema element at all**: they are the endpoint’s default-sample query (`select * where {[] a ?Concept} LIMIT 100`), liveness checks (`select ?s where {?s ?p ?o} limit 1`), 72.5% are instantiated (vs. 42.3% in the universe), Virtuoso-internal test queries with `OPTION (RVR/BREAKUP)` against `no-such-*` IRIs, and bare label/dereference lookups, automated or non-informative browser traffic rather than human information-seeking, the 2019 counterpart of the infrastructure contamination we document for the raw 2013 log (Section 4.7.1, Table 8). Only **1,065 queries (13.1%) are schema-bearing**, and it is these that produce the 116 used elements and the median support is 9 (vs. 3), and only 15.8% are singletons. To check whether even the singleton tail is spurious, we drew a random sample of 100 queried singletons and looked them up: 80% carry an explicit `wdt:P279` (subclass-of) statement or a metaclass marker — i.e. they were *intentionally* modelled as classes — and manual inspection of the remainder finds mostly genuine but rare concepts and taxa (e.g. *Quercus velutina*, 21.3% organic coverage. This does not inflate coverage or concentration, schema-less queries contribute *Pinales*, *ghat*) rather than errors; outright noise (e.g. a disambiguation item) is rare. The data-glitch risk the metric inherits is therefore real but small, and concentrated in an uninstantiated long tail that actual query demand largely avoids, zero used elements, so the numerator is drawn entirely from the schema-bearing minority, but it does mean the Bio2RDF organic-2019 figures characterise a small core of genuine human queries, not the raw browser stream, and any organic/robotic behavioural contrast on Bio2RDF-2019 should be read with that in mind. We therefore restrict the query-template analysis (Section 4.9.7) to schema-bearing queries.

4.8. Organic vs. Robotic in the 2013-era Bio2RDF Log

4.7.1. Organic vs. Robotic in the 2013-era Bio2RDF Log

The LSQ-derived *Bio2RDF-all-2013* log is anonymized and cannot be split by user agent. To examine the 2013 era at the same organic/robotic granularity as the rest of the paper, we processed the *raw* Bio2RDF server access log (the upstream source of LSQ), spanning May 2013–September 2015. It contains 127.2M executed SPARQL requests (HTTP 200), each carrying a user agent and the dataset endpoint (`cu.dataset.bio2rdf.org`).

The first lesson is about what such a log actually contains (Table 8): **organic (browser) queries are only 0.09%** of executed traffic. The bulk is machine-generated and, revealingly, much of it is *infrastructure*: 24% are Bio2RDF’s

own Virtuoso engine acting as an HTTP client (federated `SERVICE` calls and dereferencing—, 99.6% of these come from a single host in the hosting institution’s network, and their bodies are single-entity label/cross-reference lookups), and a further 22% is a sustained empty-user-agent crawl from the network of the LSQ project itself, i.e. the harvesting that built the public dataset. Any usage statistic computed over the raw log without removing such traffic would largely measure the infrastructure rather than users—a concrete instance of the example-/tool-contamination concern that any log study must control for.

Table 8

Composition of the raw 2013–2015 Bio2RDF server log (127.2M executed SPARQL requests) by user-agent class. Genuine human (browser) traffic is 0.09%; about half of all traffic is the endpoint’s own Virtuoso federation/dereferencing or the LSQ harvest.

User-agent class	Share of executed requests
Programmatic tools / libraries (Java, Jena, ...)	49.8%
Virtuoso-internal (federation / dereferencing)	24.4%
LSQ-harvest (sustained empty-UA crawl)	22.4%
Search-engine crawlers	1.7%
Other / no agent	1.6%
Organic (browser)	0.09% (115,420)

Restricting to the 115,420 organic queries (61,628 unique; 50.3% syntactically valid—the rest is the non-SPARQL web spam noted in Section 5.4.3), 2013-era organic queries cover **53.6% of the 2024 Bio2RDF schema** (167/350 types, 125/195 predicates), exercising the expected biomedical core (*drugbank:Drug*, *omim:Gene/Phenotype*, *drugbank:Target*, *kegg:Pathway*, *pharmgkb:Disease*). This is far above the 21.3% of organic-2019, but the comparison must control for effort: the 2013 organic log has roughly five times as many unique queries. Rarefying both to equal sampling effort (Section 4.6), organic-2013 recovers 33.7% vs. organic-2019’s 21.3%—a residual $1.58\times$ difference that ~~is genuine, not a size artifact. The most plausible persists after controlling for sampling effort. One possible~~ explanation is architectural: in 2013 Bio2RDF exposed a *separate endpoint per dataset*, so human exploration *may have* spread across the specialized subgraphs, whereas the single unified 2019 endpoint *may have* concentrated organic queries on a narrower core. ~~This both removes the paper’s earlier inability to split. We cannot test this directly, since the endpoint architecture and the six-year time gap change together.~~

~~This analysis contributes two things. First, it supplies the organic/robotic split for the 2013 log-and-era, which the anonymized LSQ release does not permit. Second, it adds a temporal organic-comparison that reinforces the central rarefaction message.~~ comparison of organic coverage within Bio2RDF. Both reinforce the methodological point of Section 4.6: raw coverage tracks-is strongly affected by sampling effort, and-only-an-so equal-effort comparison exposes-genuine-comparisons provide a fairer basis for comparing query behavior. We emphasise that equalizing effort controls for sample size only; it does not by itself establish that a residual difference is behavioral or architectural differences.

Schema-version robustness. The coverage above is measured against the 2024 endpoint schema, but the queries hit *earlier* Bio2RDF releases, and the schema evolved substantially across them. The 2019 log queried the same release (Release 4) that the 2024 endpoint still serves, so it is already version-matched. The 2013 log is the problem: only **17.5% of the vocabulary IRIs it references appear in the 2024 schema**, because datasets were retired and per-dataset vocabularies revised across releases (38 of the 59 datasets queried in 2013 are absent from the 2024 endpoint). Moreover, the raw 2013–2015 organic log *straddles two releases*: Release 2 (2012–2013 data) until Release 3 was deployed in mid-2014 (Release 3 grew the corpus from 25 to 35 datasets, e.g. adding *chembl*, *sider*, *reactome*). We therefore recovered both schemas from Bio2RDF’s published per-dataset statistics files (Release 2: 157 types, 628 predicates; Release 3: 351 types, 1,744 predicates), split the organic log at the mid-2014 boundary (35.8K queries before, 80.5K after), and matched each part to the release it actually queried. Version alignment recovers most of the apparently “missing” vocabulary: the fraction of referenced vocabulary present in the schema rises from 17.5% (against 2024) to **48%** for the Release-2 period (against Release 2) and **62%** for the Release-3 period (against Release 3). The corresponding version-matched coverage is 53% (417/782) and 36% (761/2,095);

~~these~~. These are not directly comparable to each other or to our headline figures ~~because the release statistics~~, and we agree that ideally the same schema definition would be applied to every release. That is not possible here, and the reason is a data-availability constraint rather than a methodological choice. Our 2024 denominator is obtained by querying a live endpoint, which lets us apply the restrictive definition of Section 3.2 (vocabulary types excluding `:Resource`, and only those predicates that connect two vocabulary types). Releases 2 and 3 are no longer served by any endpoint, so the only surviving record of their schemas is Bio2RDF's published per-dataset statistics files, which enumerate all predicates and only instantiated types —. Because we cannot re-derive the restrictive definition from those files, the historical denominators are on a broader basis than the ~~restrictive type/predicate definition used for the 2024 endpoint~~. The one. The direction of the resulting bias is nevertheless known: a broader denominator can only lower the version-matched percentages, so it cannot manufacture the increase we report. The robust, directional conclusion is that the 2024 schema substantially ~~under-recognizes~~ ~~underestimates~~ historical vocabulary, so the 2024-based 2013 coverage is a conservative cross-version estimate; aligning the schema to the queried release raises, not lowers, it. We retain the 2024-schema figures in Tables 4 and 5 ~~for cross-period consistency~~ and treat the version-matched recomputation as a robustness check; the recovered Release-2 and Release-3 schemas are released with our artifacts.

4.8. ~~What Wikidata Coverage Actually Measures: Class- vs. Value-Position Usage~~ *Wikidata-specific Analyses*

We now turn to the analyses specific to Wikidata, whose item-based model raises measurement questions that do not arise for Bio2RDF: whether the entities we extract as classes are genuine schema, and whether they are referenced as classes at all.

Validity of the extracted classes. Because a Wikidata “class” is just an item used as the object of `wdt:P31/wdt:P279`, one must ask how many extracted classes are genuine schema rather than uninstantiated or one-off (possibly erroneous) edits. We characterize each class by its *support*: the number of times it is the object of `wdt:P31` or `wdt:P279` in the KG (instances plus subclasses). Of the 103,355 extracted 2017 classes, 42.3% are instantiated (≥ 1 instance), 88.2% are embedded in the subclass hierarchy, and 30.3% are *singletons* (used as a class exactly once), the elements most exposed to data glitches. Crucially, *querying selects for well-supported classes*: among the classes referenced in organic-2017 queries, 72.5% are instantiated (vs. 42.3% in the universe), the median support is 9 (vs. 3), and only 15.8% are singletons. To check whether even the singleton tail is spurious, we drew a random sample of 100 queried singletons and looked them up. Of these, 80% carry an explicit `wdt:P279` (*subclass of*) statement or a metaclass marker, meaning they were intentionally modelled as classes. Manual inspection of the remainder finds mostly genuine but rarely used concepts rather than errors: some are biological taxa (e.g. *Quercus velutina*, *Pinales*), others are specialised non-taxonomic concepts (e.g. *ghat*, a South Asian riverside step structure). Outright noise, such as a disambiguation item, is rare. Thus, data-quality issues may slightly affect the metric, but the effect appears limited and is concentrated among rare, uninstantiated classes that are seldom queried.

4.8.1. ~~What Wikidata Coverage Actually Measures: Class- vs. Value-Position Usage~~

A schema-coverage ~~metric implicitly assumes that an element counted as “used” is used as schema~~. For metric assumes that the elements it counts are referenced as classes or predicates, rather than appearing as data values. Bio2RDF ~~this holds: its vocabulary classes are never used as property values~~. For Wikidata it does not, because vocabulary classes are only ever referenced as classes, so the assumption holds there. Wikidata items, by contrast, frequently appear both as classes and as values: the same item is routinely referenced as ~~a value~~ *the value of a property* (e.g. `Q33999` *actor* as the value of `P106` *occupation*) rather than as a class. We therefore re-examine the Wikidata results by classifying every entity reference as class-position (object of `wdt:P31/wdt:P279`, or of a path involving them) or value-position (Section 3.2).

Table 9 shows the split. Among the entities our extraction counts as Wikidata *used types* (those referenced in queries that are also objects of `wdt:P31/wdt:P279` in the KG), only **70.2% of organic** ones are ever used as a class —, **29.8% are referenced solely as values** —, whereas robotic queries use 92.9% of their counted types as classes. Weighted by reference volume the contrast narrows (both query types are dominated by value references, since queries fetch values): only ~27–29% of all type-item references are in class position. ~~Two readings follow~~ *These results have two implications*. First, the published Wikidata coverage *over-counts schema usage*, most

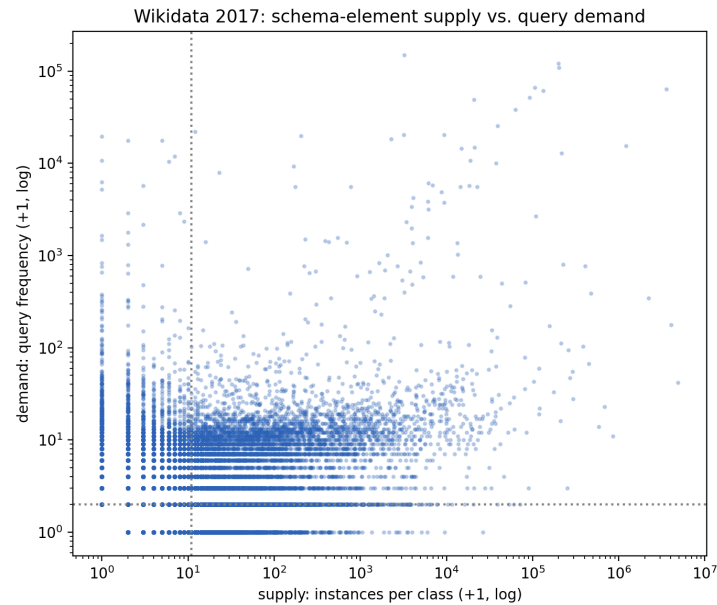


Fig. 7. Wikidata schema-element supply (instances per class, from the 2017 dump) vs. organic query demand (query frequency). Dotted lines mark the high-supply and high-demand thresholds. Heavily instantiated classes (lower right) attract little query demand, while the highest-demand items (upper left) have few or no instances because they are queried as values, not classes.

so for human queries, where roughly a third of the “used types” are mis-counted value-items (*actor, female, India*). Second, this is itself a behavioral signal: humans the class/value split differs systematically between the two query types in these logs: organic queries reference type-items as values, while bots traverse them as classes far more often than robotic ones, which reference them in class position. We report this as a difference between the analysed logs: establishing that it reflects querying behavior in general, rather than characteristics of this particular sample of clients and time period, would require further evidence.

Table 9

Class- vs. value-position usage of the Wikidata “used types” (2017). A type is used as a class only when it is the object of `wdt:P31/wdt:P279` (or a path involving them) in a query.

Query type	#Used types	Used as a class	Value-only
Wikidata organic 2017	12,773	8,968 (70.2%)	3,805 (29.8%)
Wikidata robotic 2017	58,689	54,534 (92.9%)	4,155 (7.1%)

Beyond the position of reference, comparing query demand against KG supply (instances per class, extracted from the dump) reveals a pronounced inversion (Figure 7). The most heavily instantiated Wikidata classes are almost never queried: e.g. *ravine* (~17,000 instances), *reef*, *cairn*, *qanat*, *shoal*, and (for bots) *Wikimedia navigational template* (~27,000), *protein domain*, and *microRNA* — all bulk- or bot-imported categories with essentially zero human or robotic class-position demand. Conversely, the genuinely class-queried core is small and intuitive: *human* (Q5), *city*, *film*, *country*, *painting* for organic queries, and *human-geographic territorial entity*, *film*, *album*, *church building*, *book*, *business* for robotic. In other words, the bulk of what Wikidata contains is not what users query, and much of what users query as “types” the model does not treat as classes at all.

The inversion is not idiosyncratic-confined to individual classes but holds across natural: a similar pattern is observed across the analyzed domains. We assigned each class to a top-level domain by walking up its `wdt:P279` ancestry to a curated set of anchor classes, and compared each domain’s share of KG content (instances) with its share of query demand (Table 10). The-

Two points of interpretation are needed before reading that table. First, the “supply” column is a share of *instances*, not of classes: it gives the proportion of all classified Wikidata items whose class falls in that domain. For example, *taxon* accounts for 7.6% of supply because 7.6% of all classified items are instances of some class in the *taxon* domain, not because 7.6% of classes are taxonomic. Second, the comparison is deliberately between instance-level supply and class-level demand: we count how often classes in a domain are referenced in queries, and compare that with how much data sits under those classes in the KG. The question it answers is whether users query classes in proportion to the amount of data available under them; it is not an instance-to-instance comparison, and a reader expecting one would misread the columns. The pattern is stark: the bulk-imported scientific and bibliographic domains—*taxon*, *gene/protein*, *astronomical object*, and *scholarly publication*—together hold **32% of all instances but draw only ~5% of organic demand** (*taxon* alone: 7.6% of content, 0.4% of demand). Conversely, the *person* and *organization* domains hold 15% of content but attract **~39% of organic demand**. The split also sharpens the organic/robotic contrast, and the two demand-to-supply ratio columns make it directly comparable: human queries concentrate on *person* (29.8%, ratio 2.48, vs. 14.4% and 1.20 for bots), whereas robotic queries are far heavier on *scholarly publication* (14.3%, ratio 0.75, vs. 2.4% and 0.13) and *geographic features* (21.9%, ratio 1.37, vs. 11.1% and 0.69). Notably, *geographic features* is the one domain that bots over-query relative to content while humans under-query it. This is consistent with automated bibliographic and gazetteer harvesting. (The residual “Other” bucket carries 17.9% of organic demand and includes the value-position items of Section 4.8.1, which have no clean class-domain.)

Table 10

Content–demand inversion by domain (Wikidata 2017). Each class is assigned to a top-level domain via its wdt:P279 ancestry; columns give: “Supply” is the domain’s share of KG content measured in instances (instance is the proportion of all classified items whose class lies in that domain), and of organic “Organic”/robotic “Robotic” are its shares of class-position query demand. The ratio Each demand share is organic-demand-share / supply-share accompanied by its ratio to the supply share (< 1 = under-queried relative to content, > 1 = over-queried); no ratio is given for the unclassified residual, whose supply share is not a meaningful denominator.

Domain	Supply (% instances)	Organic (%)	Org/supply	Robotic (%)	Rob/supply
Taxon / species	7.6%	0.4%	0.05	0.0%	0.00
Scholarly / publication	19.1%	2.4%	0.13	14.3%	0.75
Gene / protein / biomedical	4.9%	1.9%	0.39	1.7%	0.35
Astronomical object	0.5%	0.3%	0.60	0.1%	0.20
Creative work	33.1%	21.8%	0.66	23.3%	0.70
Geographic feature	16.0%	11.1%	0.69	21.9%	1.37
Person (human/occupation)	12.0%	29.8%	2.48	14.4%	1.20
Organization / company	3.3%	9.1%	2.76	14.0%	4.24
Event	1.2%	5.4%	4.5	3.0%	2.50
Other / unclassified	2.2%	17.9%	,	7.2%	,

Generality across modeling styles: DBpedia. A natural objection is that this An alternative explanation is that the observed value-position inflation is an artifact of usage is specific to Wikidata’s particular modeling conventions, or simply of its scale to its size, rather than of to the item-based modeling style as such. To test this we added a third KG, DBpedia, and ran the same class- vs. value-position analysis over its public query log (LSQ 2.0). DBpedia is a useful control: like Bio2RDF it is OWL-style—its classes are dedicated ontology IRIs (dbo:) used as the object of rdf:type/rdfs:subClassOf—but like Wikidata it is large, broad, and crowd-curated, so it separates the effect of schema model from that of size and breadth. Across 4.26M distinct executed queries, of the 722 dbo: classes referenced anywhere, **675 (93.5%) are used in class position and only 47 (6.5%) are value-only**—close to Bio2RDF’s near-zero and far below Wikidata’s 29.8% for human queries (Table 11). These results suggest that the value-only fraction thus tracks the data model, not is more closely associated with the data model than with KG size: the across the three KGs analysed, the two OWL-style KGs ones both keep classes in class position, whereas the item-based Wikidata routinely surfaces type-items as values. The content–demand inversion also recurs on DBpedia: its most heavily instantiated classes (*Tenure*, ~1.3M instances; *WikimediaTemplate*, ~223k; *Military-Service*; *Academic*) draw essentially no class-position demand, while queries concentrate on a small intuitive core

(Place, Person, Film, Company, MusicalArtist). ~~The phenomena we report are therefore not Wikidata-specific~~ These findings indicate that the phenomenon is not unique to Wikidata.

Table 11

Value-position inflation tracks the schema model, not KG size. Among the ontology classes *referenced in queries*, the fraction ever used in class position (object of the type/subclass property) vs. used only as a value. DBpedia and Bio2RDF are OWL-style; Wikidata is item-based.

KG (queries)	Schema model	Used as a class	Value-only
Bio2RDF 2019 (organic)	OWL classes-with-instances	~100%	~0% (by construction)
DBpedia (all executed)	OWL ontology (dbo:)	675 (93.5%)	47 (6.5%)
Wikidata 2017 (organic)	item-based	8,968 (70.2%)	3,805 (29.8%)

These observations reframe the coverage results for Wikidata: a single “schema coverage” percentage, borrowed from the OWL-style Bio2RDF setting, conflates class usage with value usage on an item-based KG. ~~A faithful figure restricts~~ Restricting coverage to class-position references over instantiated classes ; ~~the residual — and more informative — story is the structural mismatch between a KG’s instantiated content and actual query demand, which holds across both modeling styles~~ provides a more faithful estimate of schema usage. Separately, our analysis reveals a broader pattern: in all three KGs examined, the distribution of instantiated content differs substantially from the classes that users actually query. We return to the implications in Section 5.

4.9. Knowledge Graph Usage Patterns

Thus far, we have introduced a method for calculating SPARQL schema coverage, which quantifies interactions with KGs through SPARQL queries. This measure may provide only a superficial view of KG usage. Knowing that a schema element was queried does not necessarily indicate its frequency of usage or temporal consistency. In this section, we present the results of the methods described in Section 3.6, focusing on the frequency distribution and temporal changes in KG usage. For this analysis, we have used unique queries.

Basis of the frequency analyses, and its robustness to the corrected preprocessing. All frequency-based results in this section (Figures 8 and 9, Table 12, and the Wilcoxon and Spearman statistics) are computed from the released per-element usage-count files rather than recomputed from the logs. Because the corrected parameter stripping of Section 3.3 substantially raises the number of *valid* queries for Bio2RDF 2019 (Table 3), it is reasonable to ask whether it also changes these per-element counts. We therefore re-derived the per-element usage vector for *Bio2RDF-organic-2019* directly from the corrected valid-query set and compared it against the released one. The two agree closely: the Spearman rank correlation between them is $\rho = 0.954$ over the 111 shared schema elements, the ten most-used elements are identical and in almost the same order, and the largest absolute difference in any single element’s count is 21. The recomputed used-element set differs from the released one by seven elements in total (four present only in the released set, three only in the recomputed one, all of them referenced by between one and four queries), a net change of one element out of 116, which leaves coverage at 21.1% against the reported 21.3%. Two properties of this log explain why so large a change in valid-query count leaves the usage vector nearly fixed. The correction adds 11% to the valid organic queries (7,357 to 8,140), a far smaller relative gain than for the robotic log, and 86.9% of valid organic queries reference no Bio2RDF schema element at all (Section 4.7), so most of what is recovered contributes nothing to any element’s count. We therefore retain the released counts. We verified this directly for *Bio2RDF-organic-2019*, the log where the class- and frequency-level analyses are most exposed because it is by far the sparsest; the Wikidata and Bio2RDF-2013 logs were unaffected by the correction in the first place (Table 3), so their counts are unchanged by construction.

4.9.1. Frequency Distribution of Schema Elements

As described in Section 3.6.1, we analyze schema element usage patterns by visualizing the frequency distribution of elements across various intervals of robotic and organic query logs from Bio2RDF and Wikidata. Figure 8 illustrates this distribution on logarithmic axes: the y-axis is the normalized monthly usage count of each schema element and the x-axis is the schema-element rank (most- to least-frequently used). Both axes are log-scaled with natural-value tick labels. The Bio2RDF contain 545 schema elements, whereas Wikidata 2017 and 2018 include

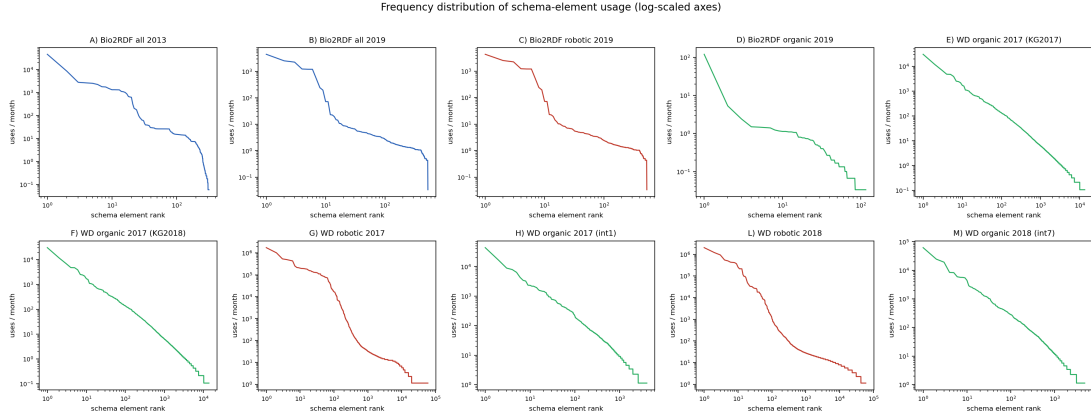


Fig. 8. Frequency distribution of schema elements across datasets (A–M). The datasets include: A) Bio2RDF All log2013_kg2024, B) Bio2RDF All log2019_kg2024, C) Bio2RDF robotic log2019_kg2024, D) Bio2RDF organic log2019_kg2024, E) Wikidata organic log2017_kg2017, F) Wikidata organic log2017_kg2018, G) Wikidata robotic log2017_kg2017, H) Wikidata organic log2017_kg2017, I) Wikidata robotic log2018_kg2018, M) Wikidata organic log2018_kg2018. The plot's color represents its log type: blue for all logs, red for robotic logs, and green for organic logs.

104,314 and 98,462 elements, respectively. The color scheme distinguishes log types: green represents organic logs, red represents robotic logs, and blue represents all logs (organic and robotic combined). The plots (A–M) exhibit a consistent trend: a sharp decline in schema element usage frequency, suggesting that most schema elements are infrequently used.

Plot A (Bio2RDF All log2013_kg2024) exhibits a more gradual decline in schema element usage compared to Plot B (Bio2RDF All log2019_kg2024). Plot C (Bio2RDF robotic log2019_kg2024) demonstrates a higher maximum frequency, with a log count approaching 4 its most-used element approaching 10^4 uses per month, whereas Plot D (Bio2RDF organic log2019_kg2024) peaks at around 2, around 10^2 . This contrast highlights the greater intensity of schema element usage in Bio2RDF robotic queries.

Plots E and F (Wikidata All organic log2017_kg2017 and Wikidata All organic log2017_kg2018) show a similar distribution of schema element usage across two different versions of Wikidata. Both intervals of robotic logs in Wikidata, Plot G and Plot L (Wikidata robotic log2017_kg2017 and Wikidata robotic log2018_kg2018), exhibit steep declines in schema element usage, with maximum frequencies exceeding a log count of 6, 10^6 uses per month. In contrast, Plot H (Wikidata organic log2017_kg2017) and Plot M (Wikidata organic log2018_kg2018) peak around 10^5 , indicating less intensive schema element usage in organic queries.

Across all datasets, both organic and robotic queries display a sharp decline in schema element usage. However, robotic queries exhibit higher initial usage, with greater peak frequencies and broader schema element usage, followed by a steeper decline.

4.9.2. Concentration of Usage: Breadth versus Evenness

Schema coverage is a *breadth* measure: it counts how many distinct elements are touched and treats an element queried once and one queried a million times alike. The frequency distributions above already suggest that this breadth is exercised very unevenly; we quantify that concentration directly so that coverage is not mistaken for uniform demand. For each dataset we compute, over the per-element usage-frequency vector (unique-query counts, the same basis as Tables 4–5), the Gini coefficient (0 = perfectly even, 1 = all demand on a single element), Pielou's evenness $J = H / \ln N$ (Shannon entropy normalised by its maximum; 1 = perfectly even), the share of all references captured by the ten most-used elements, and P_{80} , the fraction of used elements needed to account for 80% of all references (Table 12).

Usage is extremely concentrated in every dataset: the used-element Gini ranges from 0.90 to 0.99 and evenness J from 0.31 to 0.51, far below the uniform value of 1. The P_{80} column makes the long tail concrete—on the compact Bio2RDF schema 1–3% of used elements (six to nine) carry 80% of all references, and on the much larger Wikidata robotic logs the figure is below 0.1% (a few dozen of $\sim 60,000$); the ten most-used Bio2RDF elements

alone capture 83–93% of references. Because Gini and J are sensitive to N , these figures are meant for *within-KG* comparison (across query types and time), like coverage itself, not as a cross-KG ranking of a $N=116$ against a $N=60,000$ distribution. Table 12 also reports Gini and J over the *full schema universe* (the unused elements enter as zeros, so the basis is the same TSE as coverage); this is the stricter reading, since coverage concerns exactly that unused mass. Full-universe Gini is higher still (0.95–0.996), and it is where the sparsest logs move most—Bio2RDF-organic-2019 rises from 0.90 to 0.98 and Wikidata organic from 0.96 to 0.99—because most of the schema is never touched. A high coverage figure such as Bio2RDF’s 97% must therefore be read as “almost every element is touched at least once,” *not* as “demand is spread across the schema”—the two readings are nearly opposite. Coverage and concentration are thus complementary: the former measures how much of the schema is reached, the latter how unequally that reach is distributed, and a faithful usage report needs both. The slightly higher used-element evenness of the Wikidata organic logs ($J \approx 0.51$) than the robotic ones ($J \approx 0.31$ – 0.40) is consistent with the singleton-dominated robotic tail of Section 4.6.

Table 12

Concentration of schema-element usage. N = distinct used elements, TSE = total schema elements of the reference KG version. Gini and J (Pielou evenness) are computed over the per-element usage-frequency vector; top-10% = share of all references captured by the ten most-used elements; P_{80} = fraction of *used* elements accounting for 80% of references. The last two columns, Gini*/ J^* , recompute Gini/ J over the full schema universe (unused elements as zeros). Usage is highly concentrated (Gini ≥ 0.90 , and ≥ 0.95 full-universe) in every dataset; because these indices depend on N , compare within a KG, not across.

Dataset	N	TSE	Gini	J	top-10%	P_{80}	Gini* / J^*
Bio2RDF All-2013	317	399	0.955	0.382	83.2%	2.84%	0.964 / 0.367
Bio2RDF All-2019	529	545	0.952	0.386	92.8%	1.13%	0.953 / 0.384
Bio2RDF robotic-2019	529	545	0.953	0.384	93.1%	1.13%	0.954 / 0.382
Bio2RDF organic-2019	116	545	0.901	0.333	84.7%	4.31%	0.979 / 0.251
Wikidata robotic-2017	59,603	104,286	0.992	0.404	39.9%	0.09%	0.996 / 0.385
Wikidata robotic-2018	59,582	98,436	0.984	0.308	74.4%	0.02%	0.990 / 0.295
Wikidata organic-2017	13,394	104,286	0.957	0.506	52.7%	1.08%	0.994 / 0.416
Wikidata organic-2018	13,900	98,436	0.957	0.508	52.1%	1.09%	0.994 / 0.421

Wilcoxon signed-rank test. We applied the Wilcoxon signed-rank test, as described in Section 3.6.1, to assess the consistency in schema element usage frequency between the two intervals of query logs for both Bio2RDF and Wikidata datasets.

For Bio2RDF, comparing Bio2RDF All log2013_kg2024 with Bio2RDF All log2019_kg2024 yielded a statistic of 6250.000 ($p < 10^{-3}$), indicating a statistically significant difference in schema element usage between these intervals.

For Wikidata, the Wilcoxon signed-rank test also rejected the no-change null: the robotic logs (Wikidata robotic log2017_kg2017 vs. Wikidata robotic log2018_kg2018) gave a test statistic of 126,884,288.5 ($p < 10^{-3}$) and the organic logs (Wikidata organic log2017_kg2017 vs. Wikidata organic log2018_kg2018) a statistic of 607,273.0 ($p < 10^{-3}$).

We stress that with tens of thousands of paired elements a p -value is not, by itself, evidence of a *meaningful* shift: at this N any systematic difference, however small, is “significant,” so these tests confirm only that usage is not perfectly stationary. The magnitude and shape of change are better read from the effect-oriented analyses that follow—the rank-correlation (ρ) below, the equal-length-interval trajectory (Section 4.9.4, where most *adjacent*-window Wilcoxon tests are *not* significant), and the frequency distributions—which show that the change is concentrated at a few elements against an otherwise stable core rather than being a broad reshuffling.

Changes in Used Schema Element Ranking Using Equation 5, introduced in Section 3.6.1, we computed Spearman’s rank correlation and found weak to moderate consistency in schema element usage ranking across datasets. For the Bio2RDF logs (Bio2RDF All log2013_kg2024 and Bio2RDF All log2019_kg2024), the correlation coefficient of $\rho = 0.158$ ($p = 0.00497$) indicates a weak positive correlation. However, significant rank changes were observed for individual elements, such as kegg_vocabulary:Target (dropped 171 ranks), drugbank_vocabulary:group (dropped 167 ranks), and kegg_vocabulary:Disease (dropped 165 ranks).

For the Wikidata robotic logs (Wikidata robotic log2017_kg2017 vs. Wikidata robotic log2018_kg2018), a correlation coefficient of $\rho = 0.484$ ($p = 0$) suggests a moderate positive correlation, with notable rank changes for schema elements such as film award (Q4220917) (fell by 458 ranks), comic book album (Q2831984) (fell by 443 ranks), and novel series (Q1667921) (fell by 437 ranks).

For the Wikidata organic logs (Wikidata organic log2017_kg2017 vs. Wikidata organic log2018_kg2018), a correlation coefficient ($\rho = 0.571$, $p = 7.46 \times 10^{-167}$) indicates a moderate positive correlation, with extreme rank changes for elements such as P1811 (dropped 212 ranks), remarkable tree (Q811534) (rose 200 ranks), and P1423 (dropped 182 ranks).

These rank shifts highlight changes in user interactions with the KG. The weak correlation ($\rho = 0.158$) observed for Bio2RDF may be partially attributed to the 6-year interval. In contrast, the Wikidata logs, which had a one-year interval, showed moderate positive correlations ($\rho = 0.484$ for robotic logs and $\rho = 0.571$ for organic logs). The stronger correlations for Wikidata suggest a more consistent use of schema elements over a shorter time frame.

Adoption of newly introduced schema elements: The rank shifts above raise a

4.9.3. Adoption of newly introduced schema elements

The analyses so far measure usage against a schema treated as fixed within each interval, and the rank-correlation results describe how the popularity of *existing* elements shifts. Neither speaks to elements that did not exist at the start of the period. This matters for the practical use we advocate: if a maintainer prioritises documentation and interface support by observed usage, they need to know whether newly minted vocabulary is likely to be missed because it has not yet accumulated demand. We therefore ask a complementary question of *adoption dynamics*: when new schema elements are introduced, do they become popular? We can bound this only for predicates and only weakly, so we read the result as an *upper bound on adoption speed* rather than a precise measurement. The clearest signal is Bio2RDF's Release-2→Release-3 upgrade (mid-2014; Section 4.7.1), where the added vocabulary is documented: of the **1,357 predicates Release 3 added**, only **9%** are queried at all in the 2019 log and just two reach the top 20 most-used predicates—, schema growth vastly outruns query adoption, a further instance of the content–demand inversion (Section 4.8.1). For Wikidata we have no property-creation log, so we use a proxy: the 68 predicates present in our 2018 dump extraction but absent from the 2017 one. This proxy conflates a genuinely new property with one that merely first *appeared in a triple* between dumps, and $n=68$ is small, so we do not over-read it; with that caveat, these predicates are adopted in *presence* (71% queried at least once by robotic clients in 2018, 31% by organic users) but none reaches the top 100 most-queried predicates within the year (best rank 212/933 robotic, 157/546 organic; median in the bottom 15–30% of the ranking). Both observations are consistent with, but do not by themselves establish, the reading that new schema elements are *tried* yet rarely *displace the established core* on this timescale—, the temporal counterpart of the stable-core finding. A precise study would anchor “introduced” to Wikidata’s property-creation timestamps, which we leave to future work.

4.9.4. Robustness over equal-length intervals

The Wilcoxon and Spearman results above compare only two endpoints (the 2017 and 2018 intervals), so an apparent “moderate stability” could be an artifact of which two windows were chosen. To test robustness we exploited the fact that the ICCL/Dresden release segments the organic Wikidata log into consecutive *equal-length* 28-day windows. We analyzed seven such windows spanning 2017-06 to 2018-03, holding the schema universe fixed at the 2017 dump so that differences reflect query *behavior* rather than schema evolution. Because all windows are the same length, raw counts are already time-normalized.

Three findings emerge (Table 13). First, *coverage is remarkably stable*—, between 3.68% and 4.63%—, even though query volume ranges from 192K to 896K across the windows. More queries in a window do not expand coverage, which corroborates the rarefaction result (Section 4.6) that organic coverage saturates at low sampling effort. Second, *rank stability is consistent across the whole trajectory*: the consecutive-window Spearman correlation of type frequencies stays in a narrow band ($\rho = 0.47$ – 0.51 for all six adjacent pairs), decaying only mildly to $\rho = 0.44$ over the full ~ 9 -month endpoint gap. The moderate organic stability we reported for the single 2017/2018 pair is therefore not an artifact of those two endpoints—, it holds uniformly window-to-window, with gradual long-range drift. Third, the Wilcoxon signed-rank test on per-type frequency change is *not* significant for most adjacent windows ($p = 0.08, 0.38, 0.91, 0.19$), with significant shifts only across the December–January boundary ($p =$

Table 13

Organic Wikidata schema coverage over seven consecutive equal-length (28-day) windows, against the fixed 2017 schema universe (TSE=104,289). Coverage is stable at $\sim 4\%$ despite a $4.7\times$ range in query volume.

Window (28 days)	#Queries	Used types	Used preds	Coverage
2017-06-12 – 07-09	192,330	3,559	479	3.87%
2017-07-10 – 08-06	200,726	3,486	505	3.83%
2017-08-07 – 09-03	268,464	3,360	478	3.68%
2017-12-03 – 12-30	500,339	4,269	561	4.63%
2018-01-01 – 01-28	600,767	3,977	518	4.31%
2018-01-29 – 02-25	895,767	3,991	536	4.34%
2018-02-26 – 03-25	872,555	4,300	528	4.63%

0.004 and $p = 1.7 \times 10^{-8}$): usage magnitude is largely stationary month-to-month, punctuated by occasional shifts. Finally, fourteen types appear in the top-50 of *every* window—*human*, *city*, *film*, *country*, *book*, *actor*, *human settlement*, *painting*, *business enterprise*, *airport*, *cat*, *male*, *female*, and *politician*—a far stronger demonstration of a persistent “core” than the pairwise top-50 overlaps below. Notably, several core members (*actor*, *male*, *female*) are value-position items (Section 4.8.1), reinforcing that the human-query “type” core is partly composed of items the model does not treat as classes.

4.9.5. Frequent Schema Types

As described in Section 3.6.2, we analyze the most frequently queried schema types to identify common elements that persist across different time intervals and log types. Figure 9 illustrates the top 10 schema types queried across multiple datasets, comparing robotic and organic query logs from Wikidata and Bio2RDF. The x-axis shows the total usage count on a logarithmic scale (with natural-value tick labels), while the y-axis lists the schema types. In this diagram, colors indicate log types: green for organic logs, red for robotic logs, and blue for all logs (both organic and robotic). Common schema classes across paired datasets are highlighted in bold.

Plot A and Plot B (Bio2RDF All log 2013 vs. 2019) show that four out of the ten schema types—*omim:Gene*, *wormbase:Gene*, *drugbank:Target*, and *drugbank:Drug*—remain highly frequent despite a six-year gap between the two intervals. Plot C and Plot D (robotic vs. organic Bio2RDF queries of 2019) share only two common elements: *drugbank:Drug* and *phrmgkb:Variation*.

Plot E and Plot F (Wikidata All organic log 2017 with KG versions of 2017 and 2018) reveal a notable update in schema types, with “France (Q142)” newly classified as a schema type, while the rest of the frequently used schema types remain consistent with those from 2017. Plot G and Plot H (Wikidata robotic vs. organic 2017) share only three common elements: “human (Q5)”, “film (Q11424)”, and “city (Q515)”. Plot L and Plot M (Wikidata robotic vs. organic 2018) also have three common elements: “human (Q5)”, “film (Q11424)”, and “television series episode (Q21191270)”. Notably, “human (Q5)” and “film (Q11424)” remain frequent across both 2017 and 2018 for both robotic and organic query types.

The comparison of the top 50 schema types across different intervals and query types further reveals that 24% of the elements are common between Bio2rdf 2013 and 2019, 42% overlap between robotic and organic queries in Bio2rdf 2019. For Wikidata, the overlap between robotic and organic queries is 30% in 2017 and 36% in 2018, with a 46% commonality when comparing organic queries across the two years and 32% common between robotic queries across 2017 and 2018.

Together, the analyses using both the top 10 and top 50 thresholds demonstrate that a core set of schema elements consistently drives query activity across these KGs. This consistency demonstrates the fundamental role of these schema types in shaping user interactions, regardless of temporal or query-type variations. In practical terms, this stable core is exactly the set that a documentation effort, an autocomplete feature, or a query-recommendation tool for the KG could prioritize with confidence.

Example-query contamination, measured and bounded. A natural worry for any log study is that demonstration and To assess whether example queries inflate apparent usage. We quantified this directly. We the observed schema-usage statistics, we measured their prevalence in the organic log and evaluated their effect on the coverage and frequency results. We matched the full organic log against the Wikidata Query Service example set as it stood

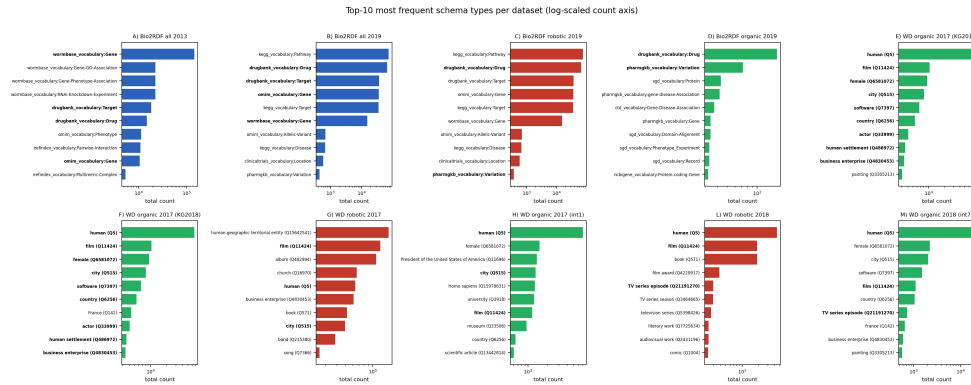


Fig. 9. The most frequent schema types across datasets (A–M). The datasets include: A) Bio2RDF All log2013_kg2024, B) Bio2RDF All log2019_kg2024, C) Bio2RDF robotic log2019_kg2024, D) Bio2RDF organic log2019_kg2024, E) Wikidata organic log2017_kg2017, F) Wikidata organic log2017_kg2018, G) Wikidata robotic log2017_kg2017, H) Wikidata organic log2017_kg2017, L) Wikidata robotic log2018_kg2018, M) Wikidata organic log2018_kg2018. The plot's color represents its log type: blue for all logs, red for robotic logs, and green for organic logs. Bold labels indicate common schema classes that appear across paired datasets side by side.

in our log period (the 2017-06-30 revision of the examples page: 348 templates, 313 of which we could fingerprint), using a variable- and literal-invariant canonical fingerprint that is robust to variable renaming and label-service boilerplate. Verbatim example runs account for only **2.5% of unique organic queries and 1.4% of executions**; removing them changes the used-type count by a single element (12,773 to 12,772) and leaves the top-type ranking unchanged. Example traffic is therefore present but does not distort the aggregate coverage or frequency results, and our use of deduplicated (unique-query) counts further limits its influence.

This measurement also ~~corrects a tempting but misleading anecdote~~ lets us settle a specific case where example-query traffic has been suspected of distorting a highly ranked schema type. The item *female* (Q6581072) appears among the most frequently queried “types” in the organic logs (Figure 9), and its apparent counterpart *male* does not, which invites the inference that the ranking is driven by the women-focused queries in the Wikidata Query Service example set. Measured directly, the asymmetry is real but far smaller than that inference supposes: *female* is referenced in 9,753 unique organic queries ; ~~its true counterpart and~~ *male* (Q6581097) is queried less, but not absent (in 2,931 unique queries) —, a ratio of roughly 3:1 , ~~not the “perfect” asymmetry one obtains by comparing female against Q6581080, which is not “male” at all but Pirhasan, an administrative quarter in Erzurum, Turkey. The residual female-skew reflects rather than the near-total absence the argument requires.~~ Since verbatim example runs account for only 1.4% of executions and their removal leaves the top-type ranking unchanged, the residual skew is not explained by example traffic that we can match. We attribute it to two factors we cannot separate with these logs: the female-weighting of the example set ~~(its women-focused queries are adapted by users , and so , whose queries users adapt and which therefore escape verbatim matching) together with genuine , and genuine research interest in gender-gap analyses on Wikidata — a content signal, not a parsing artifact. This case also . More importantly, this case~~ illustrates the class-extraction caveat of Section 3.2: *female* is counted as a “used type” only because it is an object of `wdt:P31/wdt:P279` somewhere in the data, even though in queries it functions as a property value, not a class. It is thus one of the value-position items quantified in Section 4.8.1.

Pairwise Schema-Type Usage of Bio2RDF.

4.9.6. Pairwise Schema Type Usage of Bio2RDF

~~This analysis aims to investigate how pairwise schema type patterns that represent relationships between schema types from different~~ Unlike Wikidata, Bio2RDF subgraphs have been used in queries is organized into 26 interconnected subgraphs, each a distinct source dataset with its own vocabulary. A user who wants information spanning two sources must therefore write a query that joins a type from one subgraph to a type from another. This structure gives us something Wikidata cannot offer: a direct measure of how often queries integrate across datasets, and whether that integration grew over time. We examine the evolution of this type of federated querying

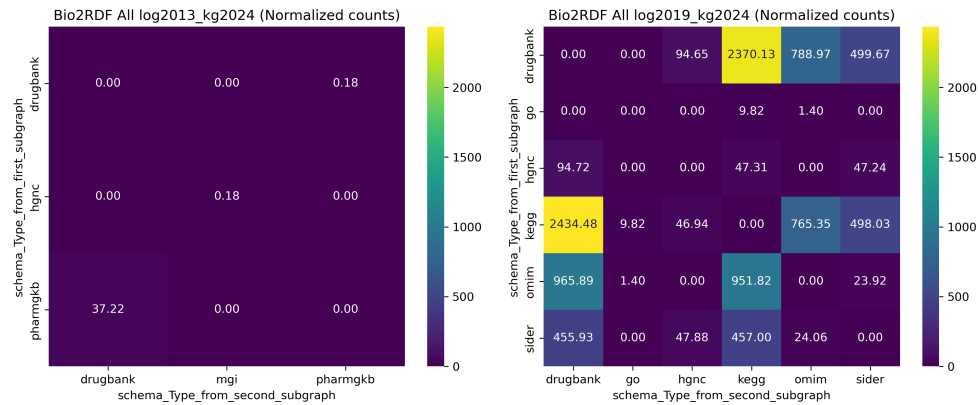


Fig. 10. Heatmap of the pairwise co-occurrence of schema types from two different subgraphs of log 2013 vs. 2019

by comparing The analysis is specific to Bio2RDF query logs from 2013 (log_2013, covering 17 subgraphs) and 2019 (log_2019, covering 26 subgraphs), as outlined in Section 3.6.2 for exactly this reason, since Wikidata has no explicit subgraph structure to join across.

A pairwise pattern here is What we measure is the pairwise schema type pattern: a query triple that links a type from one subgraph to a type from another via a (typically variable) predicate, i.e., an attempt to join data across datasets. We count how many distinct such subgraph pairs appear in each log, comparing 2013 (log_2013, covering 17 subgraphs) and 2019 (log_2019, covering 26 subgraphs), following the procedure of Section 3.6.2.

The number of distinct distinct cross-subgraph join patterns observed in queries rose from 3 in log_2013 to 22 in log_2019, roughly a sevenfold increase in federated querying over time. These two values are the counts of non-zero cells in the two heatmaps of Figure 10: each non-zero cell is one observed subgraph pair, so the 2013 panel has three populated cells and the 2019 panel twenty-two. We report this as a count rather than a fraction of “possible” patterns: the space of possible cross-subgraph joins is large (the schema exhibits 430 directed cross-subgraph type-to-type edges across 26 subgraphs) and, because observed joins use variable predicates rather than specific schema edges, a “used-out-of-possible” ratio would compare two different notions. The robust and reproducible quantity is the increase in distinct observed join patterns.

Figure 10 presents a heatmap illustrating the pairwise schema type usage across Bio2RDF subgraphs for log_2013 and log_2019. The results highlight a significant increase in the diversity of dataset integration in 2019 relative to 2013. This shift reflects broader advancements in data accessibility and integration practices within the bioinformatics community. The observed increase is likely attributable to the adoption of a single query endpoint for all datasets in 2019, which improved accessibility compared to the multiple endpoints used in 2013.

4.9.7. Which Queries Reach Rare Classes

Coverage and frequency describe which elements are used; a complementary question, related to the query-template analysis of Asprino et al. [3], is through what kinds of queries they are reached. We group queries by their predicate co-occurrence signature—the sorted set of schema predicates they reference—including property-path members included—, so that queries sharing the same schema predicates form one group. This is a much coarser notion than an Asprino template: it ignores join shape, variables, filters, and aggregation, and any two queries with the same predicates collapse together; we. The simplification is deliberate and matched to the question we ask. Our research question concerns which combinations of schema elements users reach classes through, not how complete query structures are composed; although this representation discards structural query features, it preserves exactly the schema interactions that are the focus of this study, and it has the practical advantage of being computable directly from the extracted schema elements we already have. We therefore use it only as a reproducible proxy for “kind of query,” and do not claim template equivalence. Two preprocessing points matter. First, we compute this on the human-facing logs of both KGs—, namely Bio2RDF organic-2019 and the pooled Wikidata organic windows—pooled across all seven 28-day windows of Section 4.9.4 (2017-06-12 to 2018-03-25), but restrict to schema-bearing queries (those referencing at least one schema predicate or class), because, as Section 4.5 shows,

86.9% of the Bio2RDF organic queries reference no schema element at all (endpoint default-sample/probe traffic) and would otherwise collapse into a single meaningless group; the same filter removes 23.1% of the Wikidata organic queries. This leaves **1,065** schema-bearing Bio2RDF queries and **660,672** Wikidata ones. Second, a large share of the Bio2RDF survivors (61%) are bare type-assertions (`?x a Class`) with no schema predicate, which form one high-volume group; Wikidata has none, since its classes are reached through the schema predicates `wdt:P31/wdt:P279`.

Query volume is concentrated in a modest number of signatures: **25 of 174** signatures cover 80% of the Bio2RDF schema-bearing queries, and **1,078 of 31,495** (3.4%) cover 80% of the Wikidata ones —, with a long tail of single-use signatures (63% and 42% of signatures, respectively). ~~Wikidata's item-based human queries are thus far more heterogeneous in their predicate combinations than Bio2RDF's. We do not compare the absolute number of signatures across the two KGs: their schemas differ so greatly in size, modeling style, and the number of queries analysed that the counts~~ (31,495 vs. 174 ~~distinct signatures~~), ~~consistent with a broad encyclopedic schema versus a compact biomedical one. The~~ are not on a common footing, and we draw no conclusion from their ratio. ~~The comparable quantity is the concentration within each log, reported above as the fraction of signatures needed to reach 80% of queries. The~~ generic “instances of a type” pattern is the single most popular Bio2RDF group and, because each such query names a different class, the ten most popular signatures already touch 86.5% of the distinct queried classes (on 72.7% of the volume); for Wikidata the top ten cover 62.5% of classes on 35.8% of volume.

The substantive finding is that ~~rare classes are reached through less-popular queries. The Spearman correlation between a class's query frequency and the~~: a class that few queries mention also tends to appear only in kinds of query that are themselves seldom issued.

Making this precise requires two quantities. ~~The frequency of a class is the number of queries that reference it. The popularity of a signature is the number of queries that share that signature, i.e. how commonly that combination of schema predicates is issued. For each class we take the popularity of the most popular signature referencing it in which it appears, and ask how that relates to the class's own frequency.~~

~~The relationship~~ is positive in both KGs (Spearman $\rho = 0.14$ Bio2RDF, $\rho = 0.40$ Wikidata), and the extremes are far apart: ~~the~~. The median bottom-decile (rarely queried) class is reached only through a signature used once (in Bio2RDF), or 50 times (Wikidata) in Wikidata, whereas the median top-decile class appears in a signature used 654 and 54,072 times. ~~Because a class's frequency mechanically lower-bounds the popularity of~~ respectively.

~~One objection must be dealt with before reading anything into this. If a class is referenced by n queries, then any signature containing it, we checked that this is not an artifact: is issued at least n times, so the two quantities are linked by definition and a positive correlation is guaranteed. The question is therefore whether the association exceeds that arithmetic floor. It does: for 84% of Bio2RDF classes and 99% of Wikidata classes, the most popular signature of a class is strictly more popular than the class itself for 84% (Bio2RDF) and 99% (Wikidata) of classes, with a median popularity-to-frequency ratio of $109\times$ and $5,407\times$ — i.e. popular classes genuinely ride. Frequently queried classes therefore sit on signatures that are popular for because many other queries too, so the association is not merely the arithmetic floor. Rare-use them too, not merely because of the queries mentioning that class. Rarely queried classes, by contrast, tend to appear only in rare, specialised queries — and specialised signatures, which is a behavioural counterpart to their low frequency.~~

Figure 12 presents an overview of the usage metadata generated by the proposed method. This approach processes SPARQL queries and KGs to produce usage metadata. The extracted metadata includes SPARQL schema coverage, the

4.10. Usage Metadata

The analyses above produce, for each (log, KG-version) pair, a schema coverage percentage, the set of schema elements used, their associated usage frequencies, and the generated schema for the respective KGs. The metadata for each dataset is available online for download⁸ in CSV and RDF formats and a usage frequency per element. Together these constitute the usage metadata this paper set out to generate. This subsection first demonstrates that the metadata has measurable downstream value, then describes the released artifact.

Usage-metadata-model

4.11. Utility: Usage-Metadata Forecasts Demand Better Than KG Content

4.10.1. Utility: usage metadata predicts demand better than KG content in this setting

The preceding analyses point to a concrete application: a type-suggestion (autocomplete) feature for a query builder, of the kind a KG maintainer might deploy. We evaluate it as a *forecasting* task to test whether the usage metadata is actually useful. We rank Wikidata schema types by their class-position usage frequency in the *early-2017* organic logs (the “training” period), and measure what fraction of the class-type demand in the *2018* organic log (the “test” period, ~8 months later) is captured by the top- k suggestions. ~~As a content-based baseline we rank the same types instead by KG supply (instances per class, from the 2017 dump).~~ Following the rest of our frequency analysis (Section 4), demand is computed over unique (deduplicated) queries, so that a single heavily repeated example or tool query cannot dominate the evaluation.

As a content-based baseline we rank the same types instead by KG supply (instances per class, from the 2017 dump). This baseline needs justification, because our own results in Section 4.8.1 predict that it should perform poorly, and because there is no reason in principle why the number of instances under a class should track interest in it, least of all on a KG whose content arrives through decentralized contribution and bulk import. That is precisely why we chose it. Supply is the ranking a maintainer can compute without any log access, from the KG alone, and it is therefore the default that a documentation or autocomplete effort falls back on when usage data is unavailable. The comparison is not offered as a test of a plausible rival predictor: it quantifies what is lost by using the readily available content signal in place of usage metadata, and it turns the content–demand inversion from an observation into a measurable cost.

Table 14

Forecasting utility: % of 2018 organic class-type demand (deduplicated) captured by the top- k suggested types, ranked by past (2017) usage vs. by KG supply (instances/class). Usage metadata transfers across time and outperforms the content-based ranking at every k .

Top- k suggestions	Ranked by past usage	Ranked by KG supply
10	40.3%	32.1%
50	54.1%	42.3%
100	61.7%	49.0%
500	75.7%	62.5%
1000	79.9%	68.1%

Figure 11 and Table 14 show the result. A list of just the 50 most-used types, learned from 2017, covers **54%** of the distinct-query type demand eight months later, rising to 80% at top-1000—, usage metadata transfers across time well enough to drive a real feature. The content-based ranking is markedly worse: its top-50 covers only **42%**, and it must offer **148 suggestions** ($3\times$ as many) to match what the usage ranking achieves with 50. The reason is the content–demand inversion of Section 4.8.1: the highest-supply classes are bulk- and bot-imported categories (*scholarly article*, *Wikimedia category*, *taxon*, *disambiguation page*, *Wikimedia template*, *gene*, *protein*) that humans rarely query, so a content-ranked tool squanders its top slots, whereas the usage-ranked list surfaces the intuitive core (*human*, *film*, *city*, *country*, *human settlement*, *university*, *business*). ~~This is direct evidence that the usage-metadata we generate has predictive value that KG content statistics alone do not.~~ Top- k coverage answers “how much future demand does this list capture,” but it is insensitive to the order of items within the list, which is exactly what an autocomplete dropdown exposes to the user. We therefore add an order-sensitive evaluation over the same split (Table 15): nDCG@ k , with graded relevance given by each type’s deduplicated 2018 demand, and a demand-weighted mean reciprocal rank, $MRR_w = \sum_t (d_t/r_t) / \sum_t d_t$, where d_t is the 2018 demand for type t and r_t its rank in the suggestion list. The ordering result is sharper than the coverage result. The usage ranking achieves nDCG@10 of **94.9%** against **44.5%** for the supply ranking, and stays above 92% out to $k = 1000$, meaning it places the heavily demanded types very close to the ideal order. The supply ranking never exceeds 50%: it both omits types and mis-orders the ones it includes. Demand-weighted MRR tells the same story (0.338 vs. 0.117, a $2.9\times$ difference). This matters for the application, since an autocomplete list is read from the top down.

Table 15

Order-sensitive evaluation of the type-suggestion task, same train/test split as Table 14. $nDCG@k$ uses graded relevance equal to the type's deduplicated 2018 demand; MRR_w is demand-weighted mean reciprocal rank over the full ranking. Coverage columns are repeated from Table 14 for comparison.

Top- k	$nDCG@k$ (past usage)	$nDCG@k$ (KG supply)	Coverage (usage)	Coverage (supply)
10	94.9%	44.5%	40.3%	32.1%
50	94.1%	46.3%	54.1%	42.3%
100	93.8%	47.2%	61.7%	49.0%
500	92.7%	48.6%	75.7%	62.5%
1000	92.5%	49.3%	79.9%	68.1%
MRR_w : past usage 0.338 KG supply 0.117 (ratio $2.9\times$)				

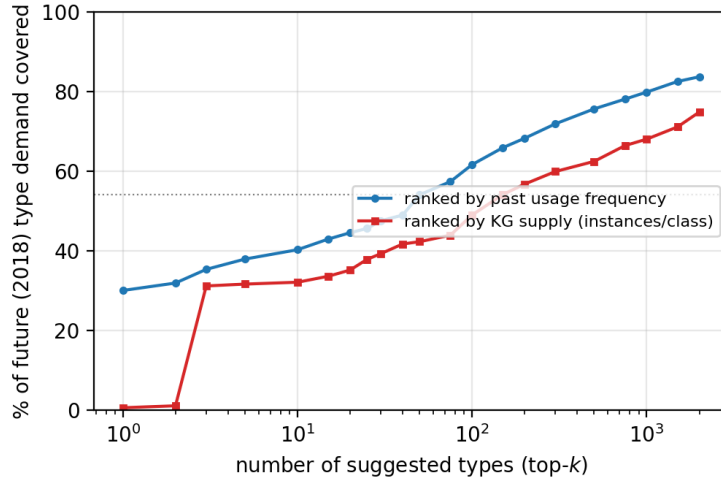


Fig. 11. Forecasting future query demand with usage metadata. Each curve shows the percentage of 2018 organic class-type demand (deduplicated) covered by the top- k suggested types, when types are ranked by their 2017 usage frequency (blue) vs. by KG supply in instances per class (red). The usage ranking dominates at every list length; the dotted line marks its top-50 coverage.

We are careful about what this demonstrates. The experiment covers one KG, one query type, and one temporal split, so it shows that usage metadata was predictive *in this setting* and that it beat the content-based default there. It does not establish that usage metadata forecasts demand in general; confirming that would require further KGs and further time periods, which we identify as future work in Section 5. What the experiment does support is the narrower and more useful claim that a usage-ranked suggestion list outperformed the content-ranked list a maintainer would otherwise reach for, and it makes concrete the documentation- and autocomplete-prioritization argument of Section 4.9.4. The advantage is well outside sampling noise: a query-level bootstrap (2,000 resamples of the 185,443 test queries) gives 95% confidence intervals of about ± 0.5 points on each coverage value, and the usage-minus-supply gap is positive at every list length—e.g. +11.8 points at top-50 (95% CI [11.2, 12.4]) and +11.8 at top-1000 ([11.3, 12.4]). Two baselines frame these numbers. A *random* ranking of the tens of thousands of candidate types would capture demand roughly in proportion to list length ($\approx k/N$, i.e. well under 3% at top-1000), so both rankings are far above chance and the informative comparison is usage-vs-supply, not usage-vs-random. Conversely, past usage is naturally autocorrelated with future usage—persistence of a popular core is expected, not surprising—so the result should be read not as “usage predicts usage” but as “a usage ranking beats the content ranking a maintainer would otherwise reach for.” Finally, because demand here is deduplicated, this measures coverage of *distinct query intents*; an autocomplete optimising raw request volume would weight by execution count, a variant we expect to favour the usage ranking even more (its core types are also the highest-volume ones).

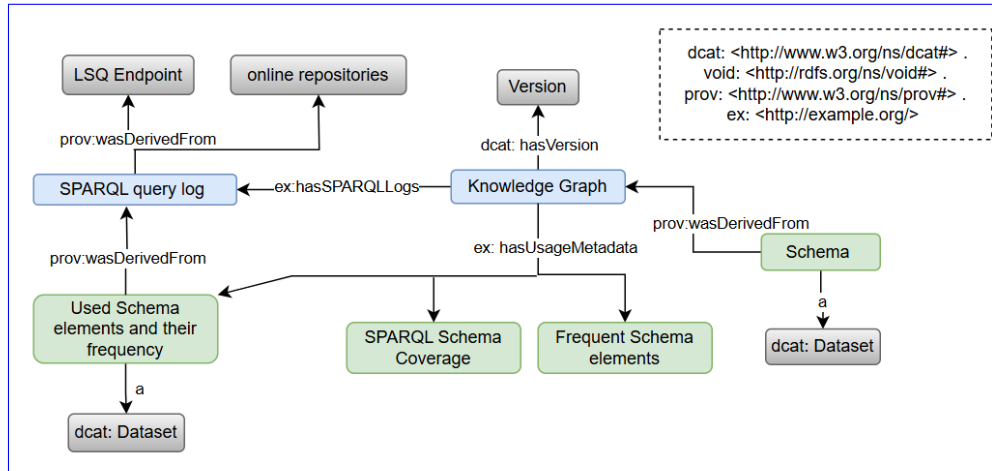


Fig. 12. Usage metadata model

4.10.2. The released usage-metadata artifact

Figure 12 presents an overview of the usage metadata generated by the proposed method. This approach processes SPARQL queries and KGs to produce usage metadata. The extracted metadata includes SPARQL schema coverage, the schema elements used, their associated usage frequencies, and the generated schema for the respective KGs. The metadata for each dataset is available online for download⁸ in CSV and RDF formats.

5. Discussion

In this study, we analyzed SPARQL query logs to explore how users interact with knowledge graphs (KGs) by addressing two key questions: (1) How can we quantify the usage of a KG? and (2) Do frequency distribution and ranking analyses of used KG elements reveal usage patterns over different log intervals or across organic (human-driven) and robotic (bot-driven) query types? We applied the proposed method to the both robotic and organic query logs from Bio2RDF and Wikidata, uncovering several patterns in schema element usage.

To quantify the overall usage of a KG, we introduced the SPARQL schema coverage metric, which measures the proportion of schema elements that are explicitly referenced in queries. Raw coverage differs markedly between robotic and organic logs (robotic 97% for Bio2RDF and 60% for Wikidata; organic 21% and 14%), which might suggest that automated processes simply access a broader range of the schema. However, robotic and organic logs also differ in size by up to two orders of magnitude, so raw coverage conflates breadth of interest with sheer query volume. The size-controlled rarefaction analysis (Section 4.6) disentangles-separates the two, and the conclusion is KG-dependent rather than outcome differs between the two datasets rather than supporting a blanket statement about “machine vs. human” querying. For In Bio2RDF, robotic logs a relatively compact biomedical KG, the robotic logs still cover roughly 1.8× more of the schema than organic logs even at equal sampling effort the organic logs at equal sampling effort, so the broader robotic footprint there is a genuine behavioral difference (systematic, repeated traversal of a compact, specialized schema). For Wikidata, the gap coverage difference there is not explained by sample size alone. In Wikidata the difference essentially vanishes at equal effort, and organic queries are even richer per query richer per occurrence at small samples; the large raw gap is mostly a volume artifact largely attributable to volume, amplified by a long tail of one-off references (two thirds of robotic Wikidata elements are used exactly once). The practical lesson is that We draw the methodological lesson rather than a structural one: schema coverage must be interpreted relative to sampling effort and to KG scale, and that “robotic queries cover more” holds for a small specialized KG but not for a very large one.

⁸https://github.com/MaastrichtU-IDS/KG_usage_metadata/tree/main/metadata-files

A further difference between the two query types is *syntactic*, and it reinforces the organic/robotic distinction on an axis independent of schema breadth. In the Bio2RDF 2019 logs, robotic queries are almost entirely well-formed, standards-compliant SPARQL (about 99.5% of unique robotic queries parse), whereas organic logs are far noisier: a large fraction of unique organic entries are not valid SPARQL at all but injected non-SPARQL text (e.g., blog-comment spam submitted to the public endpoint), and the genuinely non-standard queries that remain—using Virtuoso extensions such as `bif:contains` (full-text search), `OPTION`, and the `LIKE` operator—originate overwhelmingly from organic (human, web-form) traffic rather than from automated clients. In other words, the vendor-specific full-text and option syntax is a human behavior here, not a robotic one. This also cautions that “valid query” counts are sensitive to both log hygiene (non-SPARQL contamination of organic logs) and to engine-specific syntax; we verified that the validity rate itself is not an artifact of the chosen parser, since three independent SPARQL 1.1 parsers (a strict grammar, the parser used here, and a Rust engine) agree on validity for 98–99% of queries and a raw coverage gap between query types cannot be read as behavioral without an equal-effort comparison. With two KGs we cannot attribute the differing outcomes to schema scale as such, since the two systems also differ in domain, user community, and workload; whether the pattern tracks scale is a question for the broader study we call for in Section 5.

Beyond coverage, we observed a pronounced long-tail distribution in the frequency of schema element usage in both datasets. While a few schema elements are queried very frequently, the vast majority are rarely used. We made this precise with concentration statistics (Section 4.9.2, Table 12): the Gini coefficient of usage exceeds 0.90 in every dataset and as little as 0.1–4% of used elements account for 80% of all references. This matters for how coverage should be read: a high coverage number reports that nearly every element is touched at least once, which is almost the opposite of demand being spread evenly across the schema. This long-tail phenomenon is significant because it demonstrates that only a limited set of elements captures most of the user attention, despite the overall diversity in accessed elements. Schema coverage and usage concentration are therefore complementary rather than redundant—the breadth of elements accessed versus how unequally that breadth is exercised—and together they provide a comprehensive view of usage patterns without implying a direct causal relation between them. An information-theoretic or inequality measure (entropy, Gini) is the right summary when the question is “how concentrated is demand,” and coverage when the question is “how much of the schema is reached at all.”

We computed Spearman’s rank correlations based on the frequency rankings of common schema elements in dataset pairs. Over a longer time span (e.g., Bio2RDF All logs: 2013 vs. 2019), the weak correlation suggests shifting rankings over time. In contrast, moderate correlation in shorter intervals of Wikidata robotic logs (2017 vs. 2018) and Wikidata organic logs (2017 vs. 2018), indicates greater consistency in the near term.

Focusing on the most frequently queried schema types, our findings reveal a dual pattern in both the top 10 and top 50 elements. In the top 10 analysis, a small stable core, typically comprising 2 to 4 elements, appears consistently across different datasets, time intervals, and query types, while the remaining elements tend to vary. In the top 50 frequent elements, we observe an overlap of 24% to 46% common elements across different comparisons. This indicates that although a core set of schema elements remains persistently frequent (likely due to their fundamental role or inherent relevance), the majority of frequently queried elements are variable, reflecting evolving user interests or updates in data structures. Importantly, the persistent core can be leveraged in KG documentation and design, ensuring that these key elements are prioritized and optimized to better serve user needs.

In addition, the pairwise schema type co-occurrence analysis in Bio2RDF reveals that the number of distinct cross-subgraph join patterns observed in queries increased from 3 in the 2013 logs to 22 in the 2019 logs. This substantial increase in federated querying indicates an enhanced emphasis on integrating multiple datasets over time, likely driven by increase indicates that queries combined data from multiple Bio2RDF datasets more often in 2019 than in 2013. A likely contributing factor is improved accessibility, such as in particular the shift from multiple query endpoints a separate endpoint per dataset in 2013 to a single unified endpoint in 2019–2019, which removes the need to federate across endpoints in order to join two datasets. These results demonstrate the importance of designing KGs that support robust federated query capabilities, especially in data-driven fields like bioinformatics.

In conclusion, the findings offer a detailed understanding of KG usage patterns and present valuable guidance for refining KG design and query strategies to better address user needs. Together, these findings describe a consistent picture of the logs we analysed: usage is heavily concentrated on a small set of schema elements, that set is largely stable from one interval to the next, raw coverage differences between query types are driven substantially by log

size, and on an item-based KG a sizeable share of apparent class usage is in fact value usage. Each of these bears directly on how KG documentation, interface design, and deprecation decisions should be prioritised.

This study has several limitations, which also point to future work.

Comparability of the two KGs. The log collection periods differ (Bio2RDF ~30 months vs. Wikidata ~9 months) and the schemas differ in scale by roughly two orders of magnitude. We mitigate the first with time-normalized counts and the second by reporting coverage primarily within each KG and by the size-controlled rarefaction analysis; nonetheless, cross-KG coverage percentages should not be read as a direct ranking. For the within-Wikidata temporal comparison, the original analysis compared just two intervals (2017 and 2018); we additionally verified the two-endpoint that result against seven consecutive equal-length (28-day) windows (Section 4.9.4), confirming that coverage (~4%) and rank stability ($\rho \approx 0.47\text{--}0.51$) are not artifacts of the chosen endpoints/particular pair of intervals chosen. Direct comparison across KGs of similar scale remains valuable future work.

Schema coverage does not transfer cleanly from an OWL-style KG to an item-based one. This is the study’s most consequential lesson, and it emerged from extending a metric designed for Bio2RDF to Wikidata. In Bio2RDF, classes and predicates are OWL-style vocabulary terms and “coverage” is well defined. In Wikidata there is no formal class/instance separation; “classes” are ordinary items, and—as Section 4.8.1 shows—a large share of the items counted as queried types (about 30% in human queries) are referenced only as property *values*, never as classes. A single coverage percentage therefore conflates value usage with schema usage, and the Bio2RDF and Wikidata figures are not directly comparable: the lower Wikidata coverage is partly definitional, not purely behavioral. For item-based KGs, a faithful usage measure must (i) separate class-position from value-position references and (ii) restrict to instantiated classes. The content–demand inversion (Section 4.8.1, Figure 7) sharpens the point: Wikidata’s most heavily instantiated classes—bulk- and bot-imported geographic, biological, and structural categories—attract almost no query demand, while demand concentrates on a small, intuitive class core and on items queried as values. This also refines the organic/robotic contrast: human queries reference type-items as values far more than bots do (which traverse them as classes), so human “coverage” is the more inflated of the two. A third KG, DBpedia, isolates the causesize effect: it is as large and broad as Wikidata but OWL-style like Bio2RDF, and only 6.5% of its queried classes are value-only (Table 11), with the same content–demand inversion (heavily instantiated *Tenure/WikimediaTemplate* unqueried; demand on *Place/Person/Film*). Value-position inflation therefore follows the modeling style, not Across these three KGs, then, value-position inflation appears to be associated with the modeling style rather than with KG size or breadth, and the inversion-generalizes-across-content-demand inversion is present under both styles. We state this as an observation about the systems we examined: three KGs cannot establish that the association holds generally, and a KG with different conventions could well behave otherwise. Residual caveats remain—wdt:P279 has subproperties we do not follow, and a principled, instance-aware notion of “class” for Wikidata is an open problem [9, 21, 24]—but the broader message is that usage metrics must be matched to a KG’s modeling style.

Syntactic-Explicit vs. semantic (effective) usage. We measure syntactic-explicit usage: a schema element is used when it is explicitly written in a query. This is the right notion for documentation, interface, and deprecation decisions, but it is not the same as a *semantic* (effective) notion that would also credit the subclasses and subproperties a property path (e.g. `wdt:P31/wdt:P279*`) traverses at evaluation time and that also contribute to answers. As we show in Section 3.4, a naive closure-based notion is in fact *ill-behaved* on an item-based KG: because users occasionally anchor `wdt:P279*` at near-root classes such as *entity*, crediting the full subclass closure would mark ~62% of all Wikidata classes as “used,” driven by a handful of queries. A useful effective-usage measure would therefore need to attribute usage *partially* (e.g. weighting by how many traversed elements actually contribute answers, or capping anchor generality) rather than crediting the whole closure; designing such a measure is a natural and substantial extension.

Example-query contamination and weighting. “Organic” logs can be contaminated by demonstration and example queries. We measured this directly against the period’s Wikidata Query Service example set (Section 4.9.4’s frequent-types discussion): verbatim example runs are only ~1.4% of executions and do not change the coverage or top-type results. This is a lower bound—it does not capture example queries that users *adapt* before running (e.g. swapping the occupation or class in a template), which a fuzzier, subpattern-based matcher could estimate. Complementing our deduplicated (intent-diversity) counts with volume-weighted counts, and detecting adapted-example traffic, would further sharpen the behavioral interpretation.

Organic vs. robotic is a user-agent proxy, not a ground truth. The organic/robotic partition we use (Section 1) is the established user-agent-based classification of [5, 18]: browser-like agents are labelled organic, the remainder robotic. This is a coarse proxy. Notebook kernels, programmatic API clients, browser extensions, and increasingly LLM-driven agents do not map cleanly onto “human vs. bot,” and a determined client can spoof any user-agent string, so “robotic” is more precisely read as “non-browser client” than as “automated, non-human.” We mitigate over-reliance on the label in two ways—; our central conclusions are reported *within* each class and the size-controlled analysis (Section 4.6) guards against reading a raw coverage gap as behavioral—, and we note ~~an independent, syntactic corroboration that the classes really do differ in kind (that the two classes also differ syntactically in these logs, the~~ vendor-specific full-text and OPTION syntax ~~concentrates~~ concentrating in organic traffic ;Section 5)(Section 4.3), although as noted there that contrast may reflect the web-form access path rather than an inherent human/automated distinction. Still, the partition should be treated as an informative but imperfect signal, and a classification that incorporates query content or session structure is an avenue for future refinement.

Scope: an exploratory study across contrasting modeling styles. We deliberately position this work as an exploratory study across a small number of deliberately contrasting KG architectures—; the OWL-style biomedical Bio2RDF and the item-based encyclopedic Wikidata, with DBpedia as a third point that separates modeling style from scale—, rather than as a universal characterisation of KG usage. The headline patterns (extreme usage concentration, a stable queried core, value-position inflation tracking modeling style, and the content–demand inversion) are consistent across these systems, but confirming that they hold for other large public endpoints with their own conventions—, e.g. UniProt, ChEMBL, OpenAlex, Europeana, or government data portals—, and for KGs of similar scale but differing style, remains important future work that requires access to their query logs.

Toward demand-aware governance and execution-aware usage. Two extensions would tighten the bridge to KG maintenance. First, our supply–demand analysis (Section 4.8.1) identifies schema elements that are heavily populated yet rarely queried; pairing this with a *maintenance-cost* signal (e.g. update frequency, provenance complexity, or storage footprint per class) would let maintainers target review at elements that are expensive to keep current but attract little demand—, a direct input to deprecation and curation decisions. Second, our usage notion is binary (an element is referenced or not). A richer, execution-aware hierarchy—; referenced, executed, executed successfully, and actually contributing to returned results—, would distinguish these increasingly stringent notions of “use”; the LSQ-derived logs we work with do not carry result-set or success information, so this requires instrumented endpoints and is left to future work.

5.1. Implications for practitioners

Beyond the measurement contributions, our results translate into three concrete recommendations for KG maintainers. **(1) Do not rank or compare KGs by a single “schema coverage” number.** On an item-based KG the figure conflates class usage with value usage and is dominated by a barely-instantiated long tail; a faithful report separates class-position from value-position references and restricts to instantiated classes (Section 4.8.1). **(2) Drive documentation, interface, and deprecation decisions from usage rather than from content volume.** The content–demand inversion means the most-instantiated classes are often the least queried; our forecasting experiment (Section 4.10.1) shows a usage-ranked suggestion list captures markedly more future demand than a content-ranked one of equal length, and that the queried core is small, intuitive, and temporally stable (Section 4.9.4)—, exactly the set worth documenting and surfacing in autocomplete, while the persistently-unqueried, uninstantiated tail is where deprecation review can focus. **(3) Separate the human signal from the bot signal, and guard against example-query contamination.** The organic and robotic logs differ not only in volume but in what and how they query; size-controlled analysis (Section 4.6) is needed before reading a coverage gap as behavioral, and demonstration queries can masquerade as organic demand and must be detected before usage statistics are trusted for human-facing decisions.

6. Conclusion

This study introduces a novel approach for analyzing SPARQL query logs to gain deeper insights into user interactions with KGs. To identify usage patterns, we introduce the SPARQL schema coverage metric and conduct

frequency distribution and ranking analyses of queried KG elements across different log intervals and both organic and robotic query types from Bio2RDF and Wikidata. The findings reveal that both query types in both datasets exhibit a long-tail distribution in schema element usage. Raw coverage is higher for robotic queries (60–97%) than organic ones (13–21%), but a size-controlled (rarefaction) analysis shows that this gap ~~reflects a genuine behavioral difference only for the compact~~ persists after controlling for sampling effort only for Bio2RDF schema; for the much larger; for Wikidata it is largely a consequence of robotic logs being far larger. A further, central finding is that schema coverage does not transfer cleanly between KGs of different modeling styles: on the item-based Wikidata, roughly 30% of the items counted as queried “classes” in human queries are in fact used as property values, and the most heavily instantiated classes are rarely queried — a structural mismatch between KG content and query demand that a single coverage percentage obscures. A third KG, DBpedia, ~~confirms~~ indicates that this value-position inflation ~~tracks~~ is associated with the modeling style (item-based vs. OWL) rather than with KG size, while the content-demand inversion ~~generalizes across~~ is present under both styles. With three KGs these are reported observations rather than established regularities. The same modeling difference makes a naive closure-based notion of “effective usage” ill-behaved on Wikidata, where a handful of queries anchored at near-root classes would mark most of the ontology as used — evidence that explicit reference is the appropriate primitive for an item-based KG. The temporal analysis identifies a stable core of elements that consistently drive user queries over time; across seven consecutive equal-length intervals the organic coverage and the element-ranking correlation are stable, confirming that this is not an artifact of which two endpoints are compared, while over the six-year Bio2RDF span rankings shift more. Finally, we demonstrate that this usage metadata has concrete predictive value: a type-suggestion ranking learned from one year of logs captures most of the next year’s query demand and substantially outperforms a ranking by KG content, which wastes its top suggestions on bulk-imported but unqueried classes. Overall, our findings advance the understanding of KG usage patterns and provide guidance for optimizing KG design, documentation, and query strategies, ultimately contributing to the development of more responsive and user-centric knowledge graphs.

The implementation of this method is publicly available on the GitHub repository: https://github.com/MaastrichtU-IDS/KG_usage_metadata.

7. Acknowledgment

This project has received funding from the European Union’s Horizon 2020 research and innovation program under the Marie Skłodowska-Curie grant agreement No 860801.

References

- [1] B. Abu-Salih, Domain-specific knowledge graphs: A survey, *Journal of Network and Computer Applications* **185** (2021), 103076.
- [2] W. Ali, M. Saleem, B. Yao, A. Hogan and A.-C.N. Ngomo, A survey of RDF stores & SPARQL engines for querying knowledge graphs, *The VLDB Journal* (2022), 1–26.
- [3] L. Asprino and M. Ceriani, How is Your Knowledge Graph Used: Content-Centric Analysis of SPARQL Query Logs, in: *International Semantic Web Conference*, Springer, 2023, pp. 197–215.
- [4] F. Belleau, M.-A. Nolin, N. Tourigny, P. Rigault and J. Morissette, Bio2RDF: towards a mashup to build bioinformatics knowledge systems, *Journal of biomedical informatics* **41**(5) (2008), 706–716.
- [5] A. Bielefeldt, J. Gonsior and M. Krötzsch, Practical Linked Data Access via SPARQL: The Case of Wikidata., in: *LDOW@ WWW*, 2018.
- [6] A. Bonifati, W. Martens and T. Timm, An Analytical Study of Large SPARQL Query Logs, *Proceedings of the VLDB Endowment* **11**(2) (2017), 149–161. <https://www.vldb.org/pvldb/vol11/p149-bonifati.pdf>.
- [7] A. Bonifati, W. Martens and T. Timm, Navigating the Maze of Wikidata Query Logs, in: *The World Wide Web Conference (WWW)*, ACM, 2019, pp. 127–138.
- [8] A. Bonifati, W. Martens and T. Timm, An analytical study of large SPARQL query logs, *The VLDB Journal* **29**(2) (2020), 655–679.
- [9] F. Brasileiro, J.P.A. Almeida, V.A. de Carvalho and G. Guizzardi, Applying a Multi-Level Modeling Theory to Assess Taxonomic Hierarchies in Wikidata, in: *Proceedings of the 25th International Conference Companion on World Wide Web (WWW Companion)*, ACM, 2016, pp. 975–980.
- [10] C. Buil-Aranda, M. Ugarte, M. Arenas and M. Dumontier, A preliminary investigation into SPARQL query complexity and federation in Bio2RDF, in: *Alberto mendelzon international workshop on foundations of data management*, 2015, p. 196.

- [11] A. Chao, Nonparametric Estimation of the Number of Classes in a Population, *Scandinavian Journal of Statistics* **11**(4) (1984), 265–270.
- [12] J. Hammerer and W. Martens, A Compendium of Regular Expression Shapes in SPARQL Queries, in: *Proceedings of the 8th Joint Workshop on Graph Data Management Experiences & Systems (GRADES) and Network Data Analytics (NDA)*, ACM, 2025, pp. 4:1–4:10.
- [13] S. Handschuh, M. Hausenblas, R. Cyganiak and K. Möller, Learning from Linked Open Data Usage: Patterns & Metrics, 2010. <https://api.semanticscholar.org/CorpusID:15563610>.
- [14] A. Hogan, E. Blomqvist, M. Cochez, C. d’Amato, G.D. Melo, C. Gutierrez, S. Kirrane, J.E.L. Gayo, R. Navigli, S. Neumaier et al., Knowledge graphs, *ACM Computing Surveys (Csur)* **54**(4) (2021), 1–37.
- [15] S.H. Hurlbert, The Nonconcept of Species Diversity: A Critique and Alternative Parameters, *Ecology* **52**(4) (1971), 577–586.
- [16] M. Kaminski and E.V. Kostylev, Complexity and Expressive Power of Weakly Well-Designed SPARQL, *Theory of computing systems* **62** (2018), 772–809. <https://doi.org/10.1007/s00224-017-9802-9>.
- [17] J. Lorey and F. Naumann, Detecting SPARQL query templates for data prefetching, in: *The Semantic Web: Semantics and Big Data: 10th International Conference, ESWC 2013, Montpellier, France, May 26–30, 2013. Proceedings 10*, Springer, 2013, pp. 124–139.
- [18] S. Malyshev, M. Krötzsch, L. González, J. Gonsior and A. Bielefeldt, Getting the most out of Wikidata: Semantic technology usage in Wikipedia’s knowledge graph, in: *The Semantic Web–ISWC 2018: 17th International Semantic Web Conference, Monterey, CA, USA, October 8–12, 2018, Proceedings, Part II 17*, Springer, 2018, pp. 376–394.
- [19] M. Mohammadi, (Semi-) Automatic Construction of Knowledge Graph Metadata, in: *European Semantic Web Conference*, Springer, 2022, pp. 171–178.
- [20] M. Mohammadi and M. Dumontier, Analyzing SPARQL Query Logs: A Study on Schema Coverage and Query Content, in: *Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS)*, 2024. <https://ceur-ws.org/Vol-3890/paper-14.pdf>.
- [21] P.F. Patel-Schneider and E.A. Dogan, Class Order Disorder in Wikidata and First Fixes, *CoRR abs/2411.15550* (2024). <https://arxiv.org/abs/2411.15550>.
- [22] C. Peng, F. Xia, M. Naseriparsa and F. Osborne, Knowledge graphs: Opportunities and challenges, *Artificial Intelligence Review* **56**(11) (2023), 13071–13102.
- [23] F. Picalausa and S. Vansumneren, What are real SPARQL queries like?, in: *Proceedings of the International Workshop on Semantic Web Information Management*, 2011, pp. 1–6.
- [24] A. Piscopo and E. Simperl, Who Models the World?: Collaborative Ontology Creation and User Roles in Wikidata, *Proceedings of the ACM on Human-Computer Interaction* **2**(CSCW) (2018), 141:1–141:18.
- [25] A. Raghuveer, Characterizing machine agent behavior through SPARQL query mining, in: *Proceedings of the International Workshop on Usage Analysis and the Web of Data, Lyon, France*, Citeseer, 2012.
- [26] L. Rietveld, R. Hoekstra et al., Man vs. machine: Differences in SPARQL queries, in: *Proceedings of the 4th USEWOD Workshop on Usage Analysis and the Web of Data, ESWC*, 2014.
- [27] M. Saleem, M.I. Ali, A. Hogan, Q. Mehmood and A.-C.N. Ngomo, LSQ: the linked SPARQL queries dataset, in: *The Semantic Web–ISWC 2015: 14th International Semantic Web Conference, Bethlehem, PA, USA, October 11–15, 2015, Proceedings, Part II 14*, Springer, 2015, pp. 261–269.
- [28] C. Spearman, The proof and measurement of association between two things. (1961).
- [29] C. Stadler, M. Saleem, Q. Mehmood, C. Buil-Aranda, M. Dumontier, A. Hogan and A.-C. Ngonga Ngomo, LSQ 2.0: A linked dataset of SPARQL query logs, *Semantic Web* (2024), 1–23.